12 – OS & Scheduling

What is the running order of processes?

Jérôme Landré – **jerome.landre@ju.se**

OUTLINE

0) Types of Operating Systems
    - 0.1) The Digital World
    - 0.2) Internet of Things
    - 0.3) Types of Operating Systems
1) Scheduling tasks
2) Types of Schedulers
3) Preemptive Scheduling
4) Interrupts
5) Cooperative Scheduling

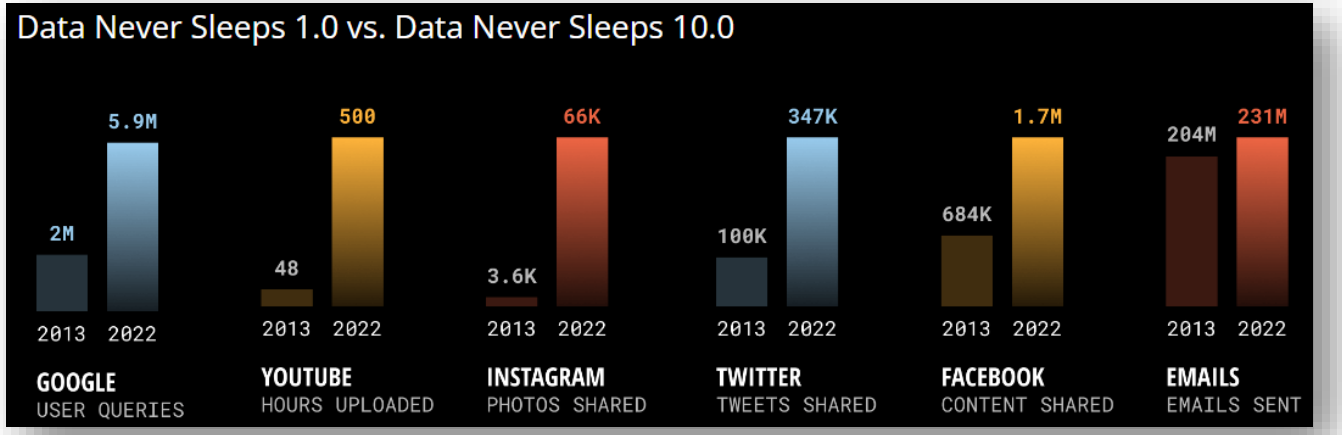
0) OPERATING SYSTEMS

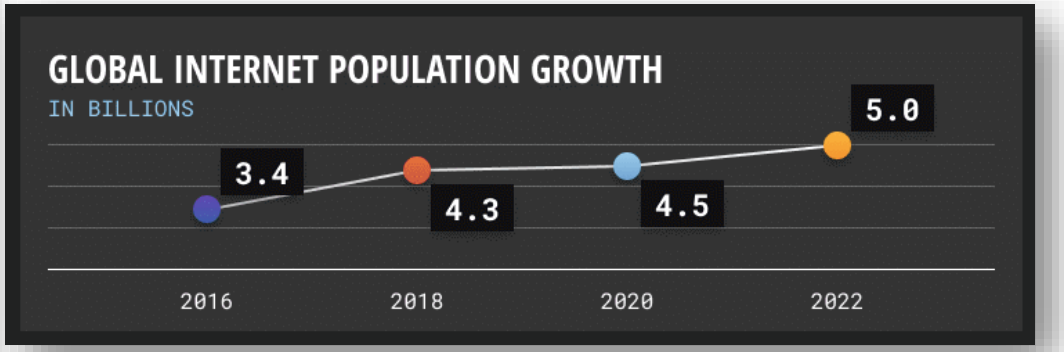
- Operating Systems are everywhere:
  - Computers,
  - Phones,
  - Lawn mowers,
  - Embedded systems,
  - Cars,
  - Satellites,
  - …

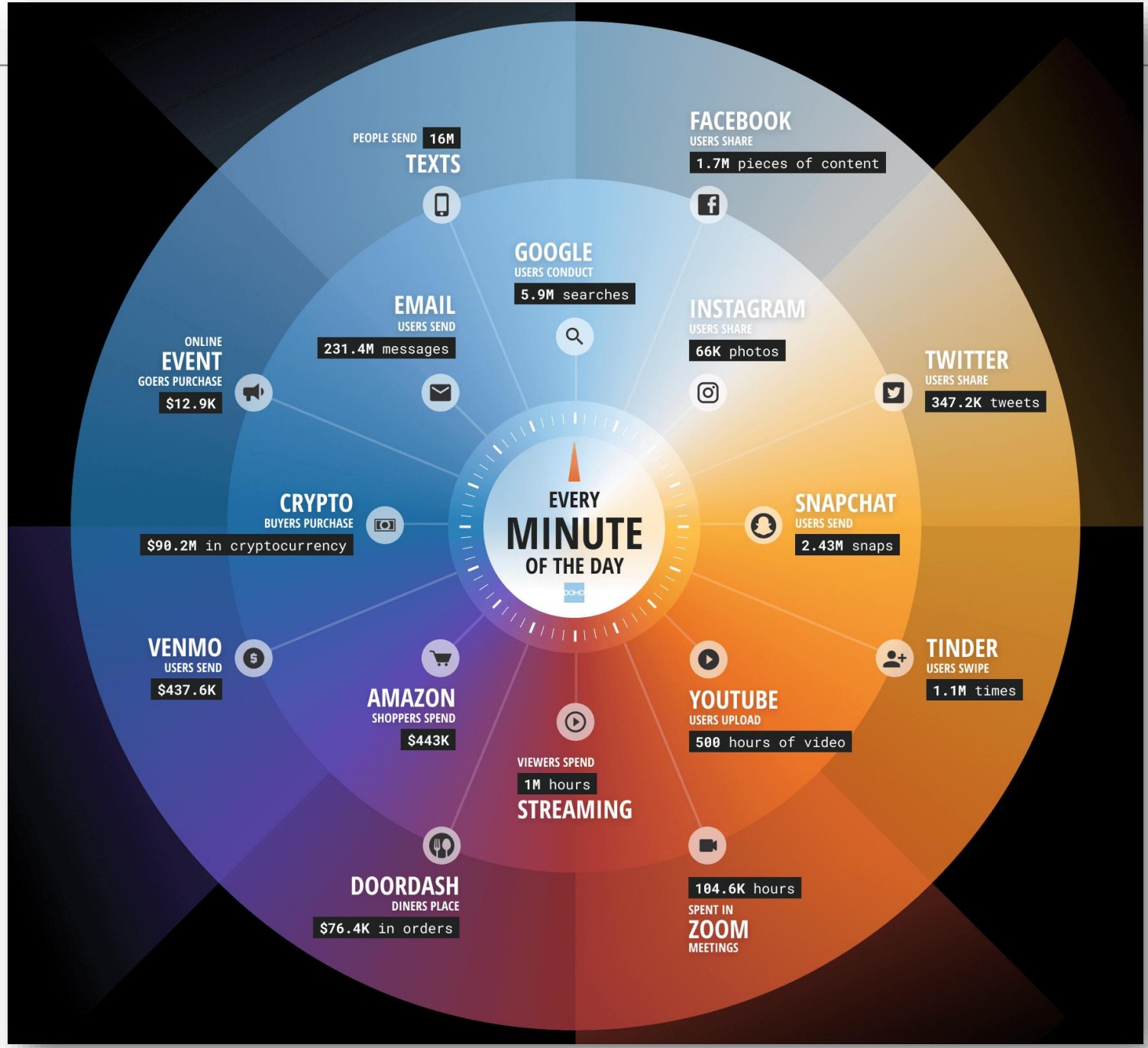
0.1) THE DIGITAL WORLD

- Our world is digital: our phones, computers, applications are used all along the day
- Every minute of the day, the amount of **data** created is just incredible:

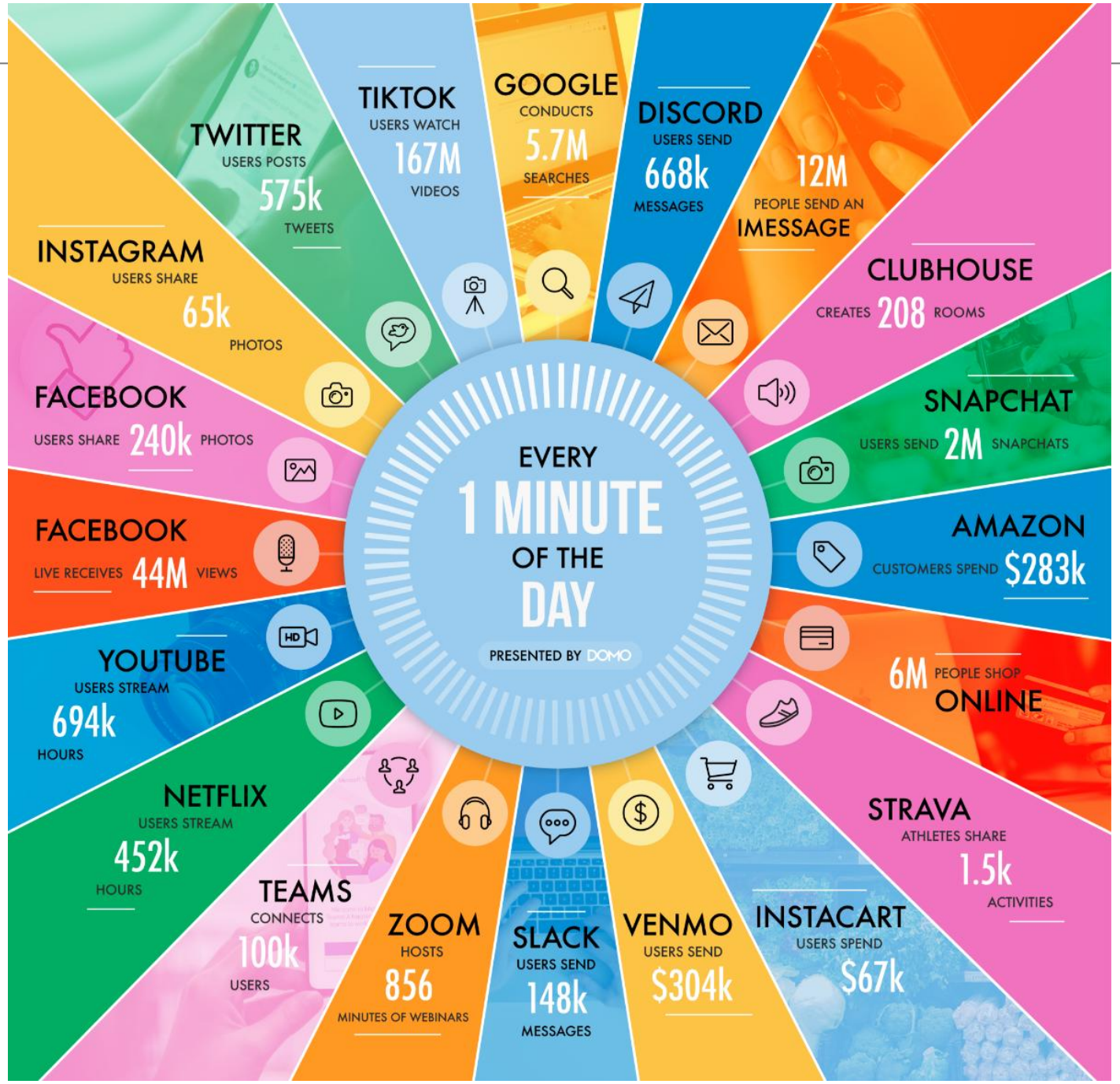
https://www.domo.com/data-never-sleeps






EVERY
MINUTE
OF THE DAY
PEOPLE SEND 16M
TEXTS
FACEBOOK
USERS SHARE
1.7M pieces of content
GOOGLE
USERS CONDUCT
5.9M searches
EMAIL
USERS SEND
231.4M messages
INSTAGRAM
USERS SHARE
66K photos
ONLINE
EVENT
GOERS PURCHASE
$12.9K
TWITTER
USERS SHARE
347.2K tweets
CRYPTO
BUYERS PURCHASE
$90.2M in cryptocurrency
SNAPCHAT
USERS SEND
2.43M snaps
VENMO
USERS SEND
$437.6K
AMAZON
SHOPPERS SPEND
$443K
TINDER
USERS SWIPE
1.1M times
YOUTUBE
USERS UPLOAD
500 hours of video
VIEWERS SPEND
1M hours
STREAMING
DOORDASH
DINERS PLACE
$76.4K in orders
104.6K hours
SPENT IN
ZOOM
MEETINGS

2022


EVERY
1 MINUTE
OF THE
DAY
PRESENTED BY DOMO
TWITTER
USERS POSTS
575k
TWEETS
TIKTOK
USERS WATCH
167M
VIDEOS
GOOGLE
CONDUCTS
5.7M
SEARCHES
DISCORD
USERS SEND
668k
MESSAGES
12M
PEOPLE SEND AN
IMESSAGE
CLUBHOUSE
CREATES 208 ROOMS
INSTAGRAM
USERS SHARE
65k
PHOTOS
FACEBOOK
USERS SHARE 240k PHOTOS
SNAPCHAT
USERS SEND 2M SNAPCHATS
FACEBOOK
LIVE RECEIVES 44M VIEWS
AMAZON
CUSTOMERS SPEND $283k
YOUTUBE
USERS STREAM
694k
HOURS
6M PEOPLE SHOP
ONLINE
NETFLIX
USERS STREAM
452k
HOURS
STRAVA
ATHLETES SHARE
1.5k
ACTIVITIES
TEAMS
CONNECTS
100k
USERS
ZOOM
HOSTS
856
MINUTES OF WEBINARS
SLACK
USERS SEND
148k
MESSAGES
VENMO
USERS SEND
$304k
INSTACART
USERS SPEND
$67k

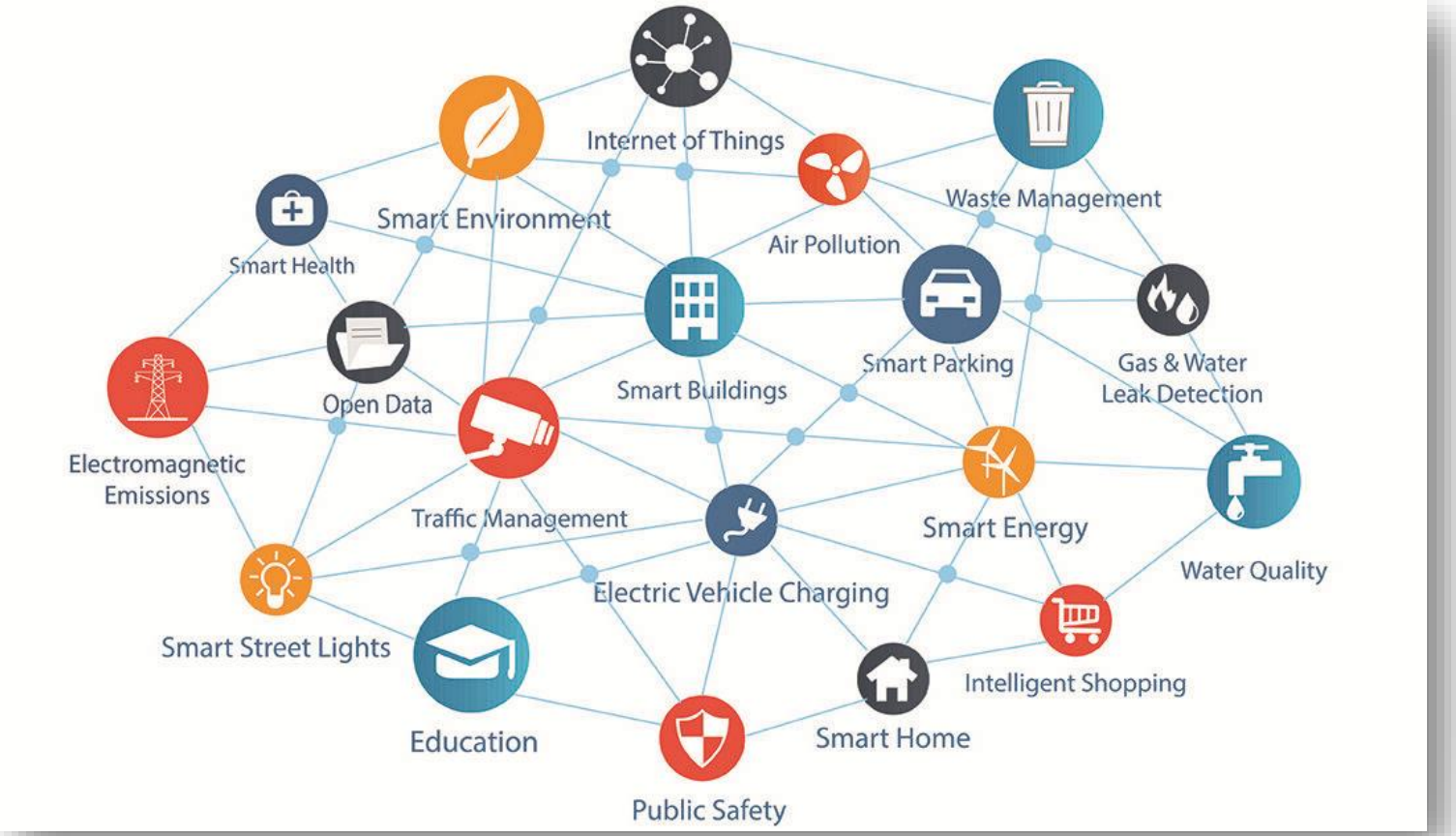
2021

0.2) INTERNET OF THINGS

- According to Wikipedia, The **Internet of things (IoT)** describes physical objects (or groups of such objects) with sensors, processing ability, software, and other technologies that connect and exchange data with other devices and systems over the Internet or other communications networks.

Source: https://www.linkedin.com/pulse/internet-things-iot-%C5%9Fahin-meri%C3%A7/, https://en.wikipedia.org/wiki/Internet_of_things

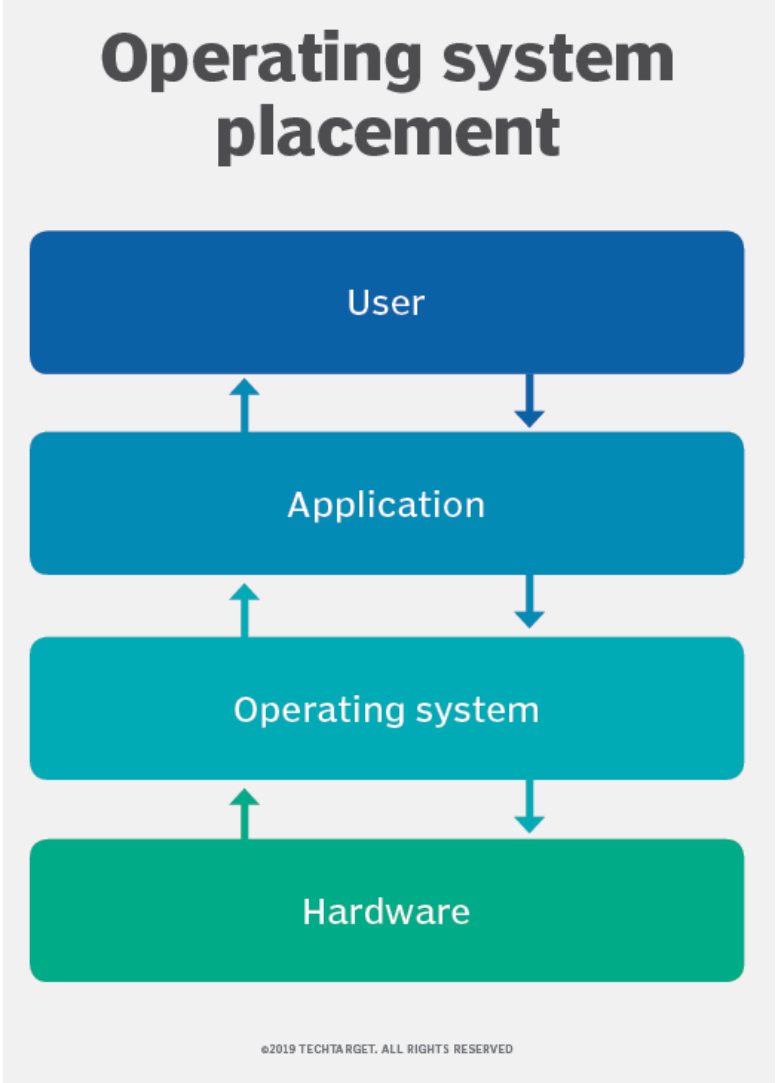
0.2) INTERNET OF THINGS



Source: https://www.nffinc.com/webinar-securing-your-internet-of-things-iot-program-three-real-world-examples-june-24-2021/, https://www.fingent.com/blog/role-of-data-analytics-in-internet-of-things-iot/

3) TYPES OF OPERATING SYSTEMS

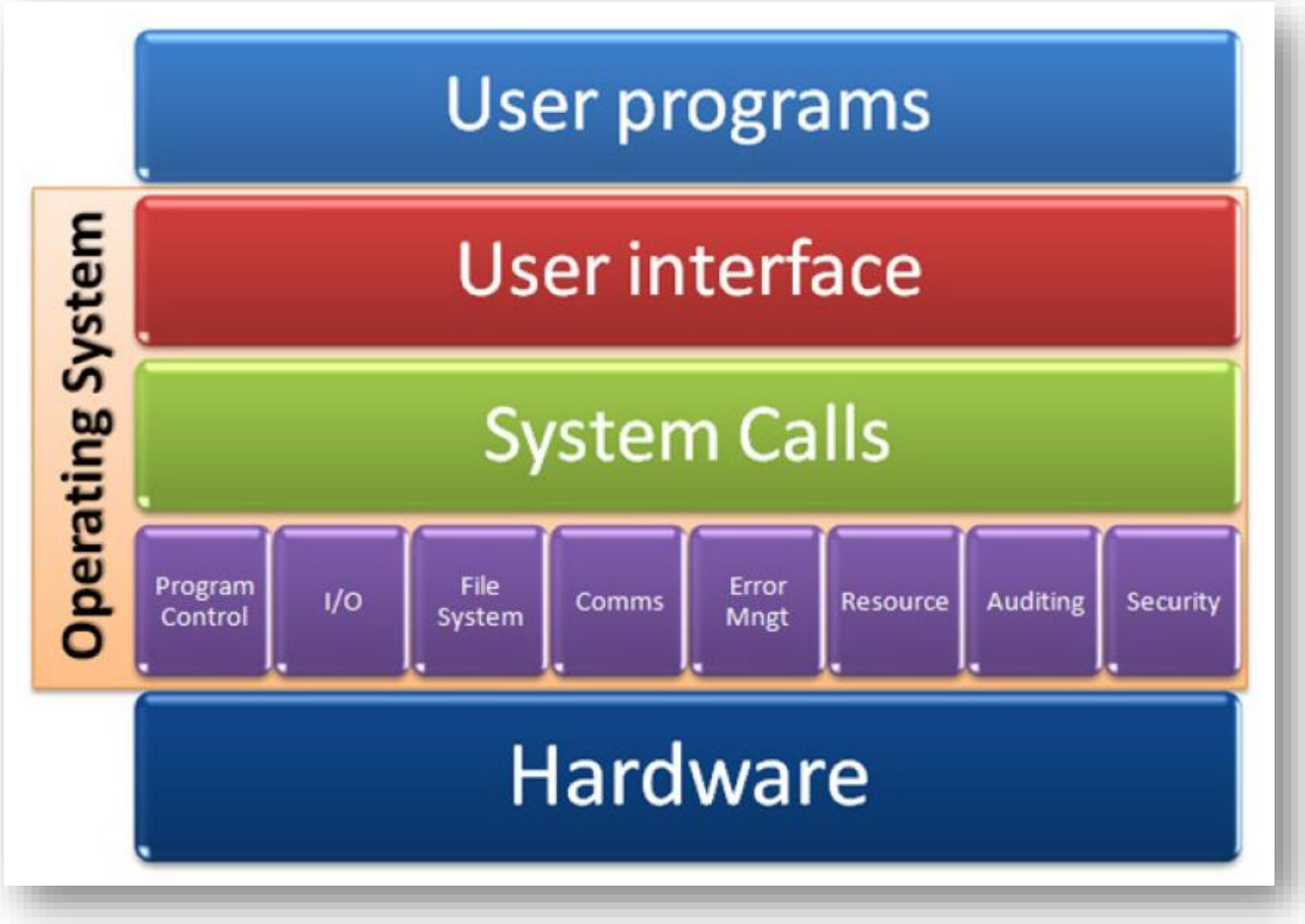
- The Operating Systems (OS) is in between the applications and the hardware
- Its role is to show to the users only the part of the hardware they can interact with



Source: https://www.techtarget.com/whatis/definition/operating-system-OS

0.3) TYPES OF OPERATING SYSTEMS

- The Operating Systems (OS) offers System Calls to applications in order to get access to the hardware
- OS accepts drivers installation to manage the capabilities of the hardware

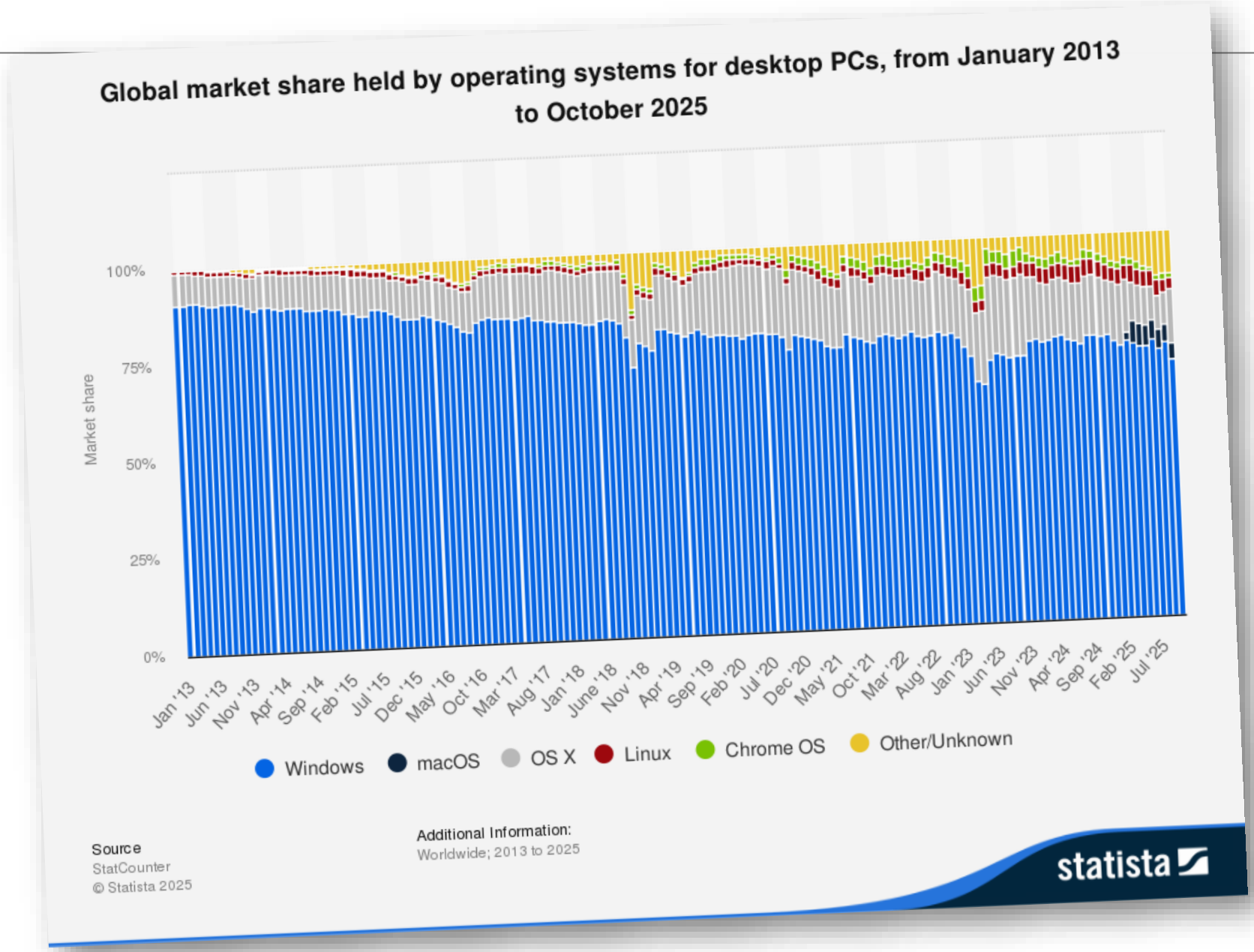


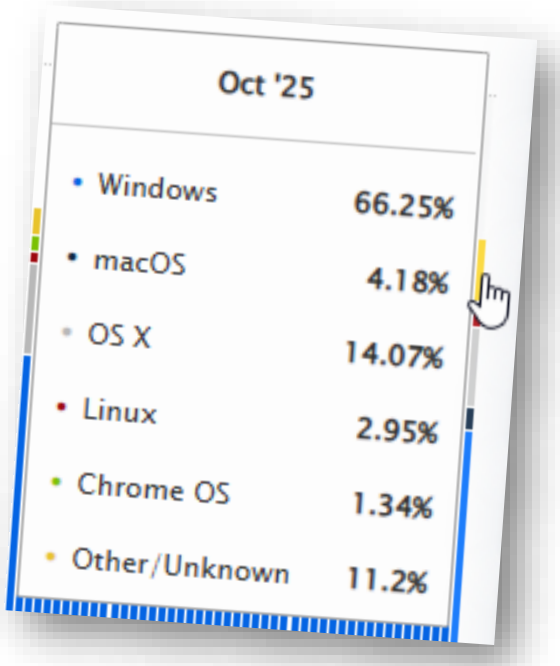
Source: Andreas AXELSSON's course

0.3) TYPES OF OPERATING SYSTEMS

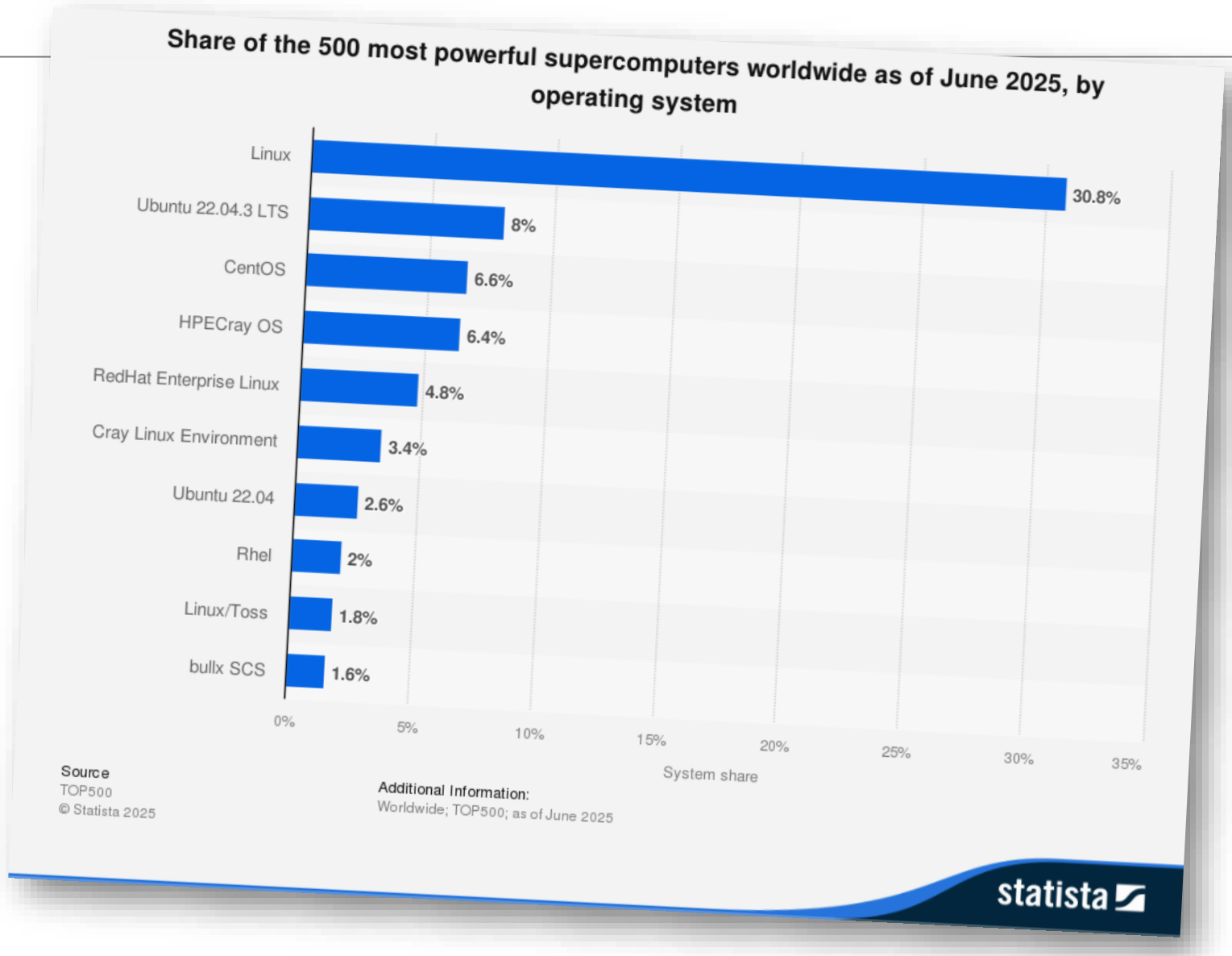
- There exists many types of Operating Systems (OS) that are specialized or generic ones:
  - **Multi-tasking OS**: programs are run concurrently
  - **Multi-user OS**: multiple users at the same time
  - **Distributed OS**: they make a networked set of computers to be seen as one computer only
  - **Network OS**: software that connects multiple devices and computers on the network and allows them to share resources on the network
  - **Embedded OS**: they operate on small (limited resources) machines
  - **Real-time OS**: they guarantee to process events in a specific moment in time

Source: https://en.wikipedia.org/wiki/Operating_system, https://www.geeksforgeeks.org/types-of-operating-systems/, https://www.geeksforgeeks.org/what-is-a-network-operating-system/

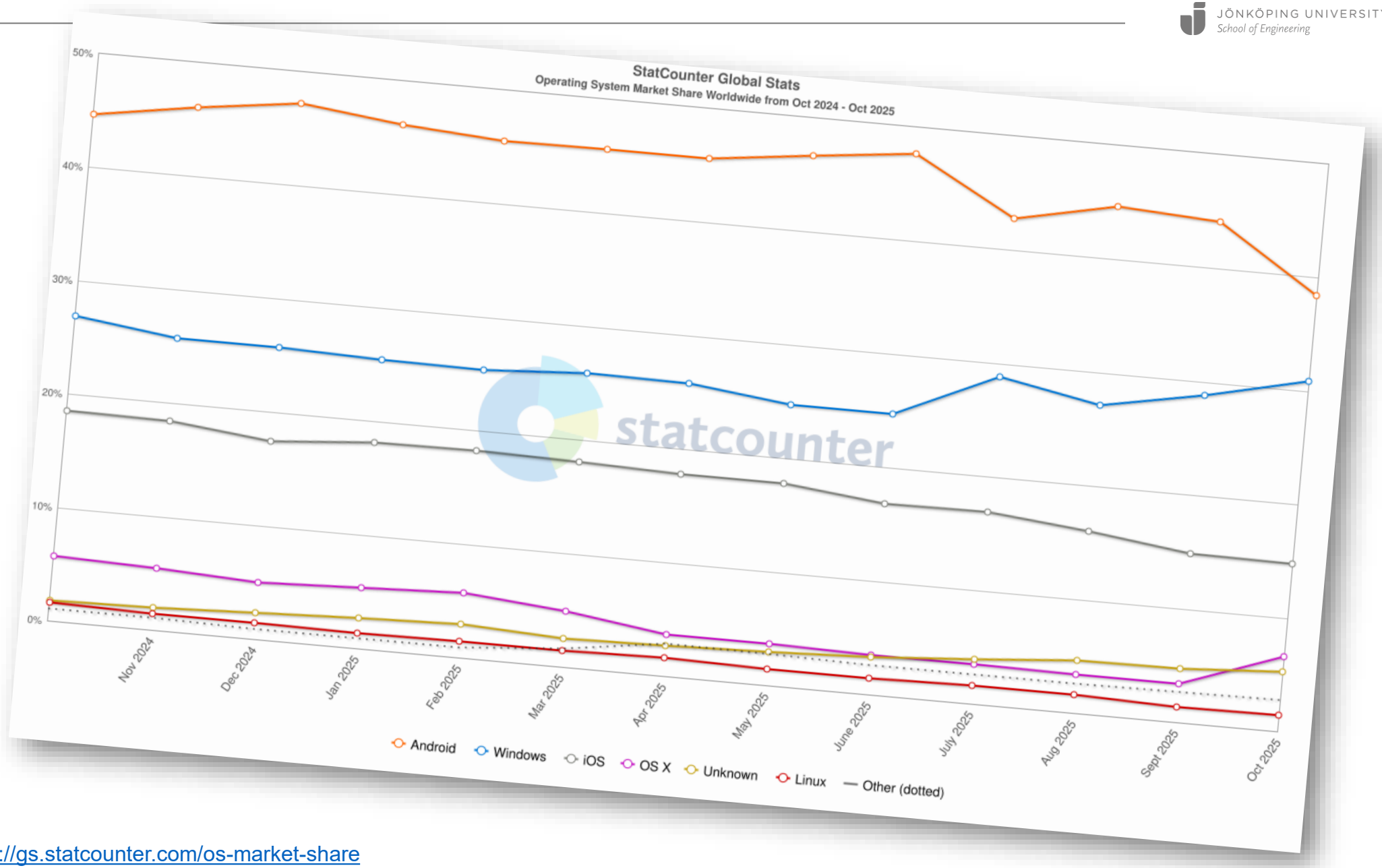




Source: https://www.statista.com/statistics/218089/global-market-share-of-windows-7/



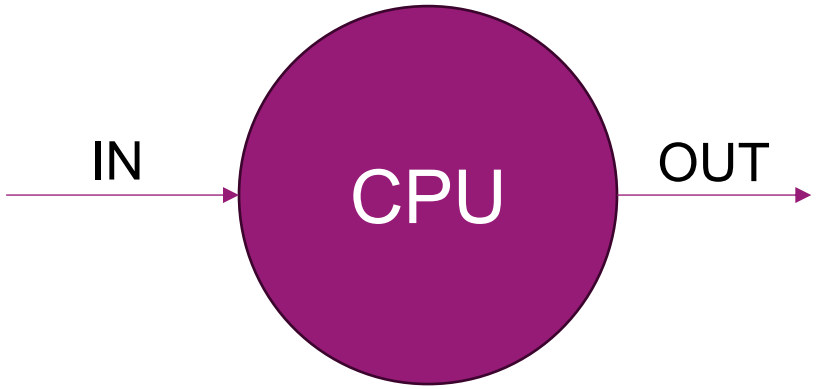
Source: https://www.statista.com/statistics/565080/distribution-of-leading-supercomputers-worldwide-by-operating-system-family/



Source: https://gs.statcounter.com/os-market-share

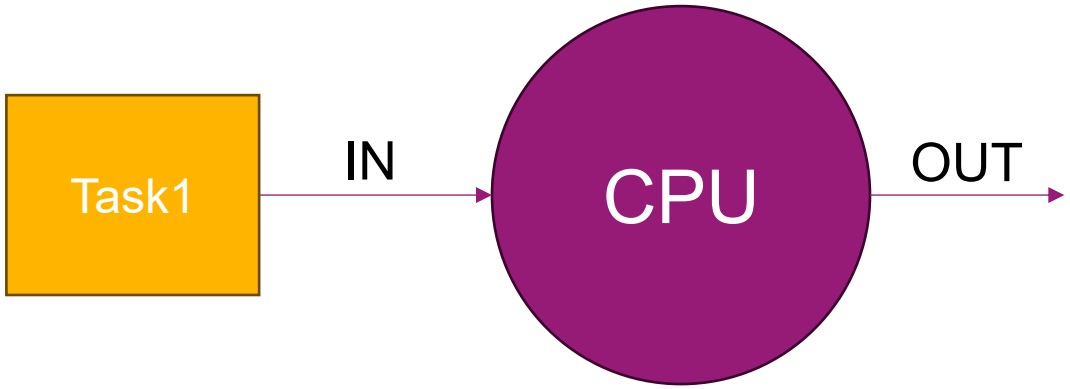
1) SCHEDULING TASKS

- If we have **one task** at a time: easy



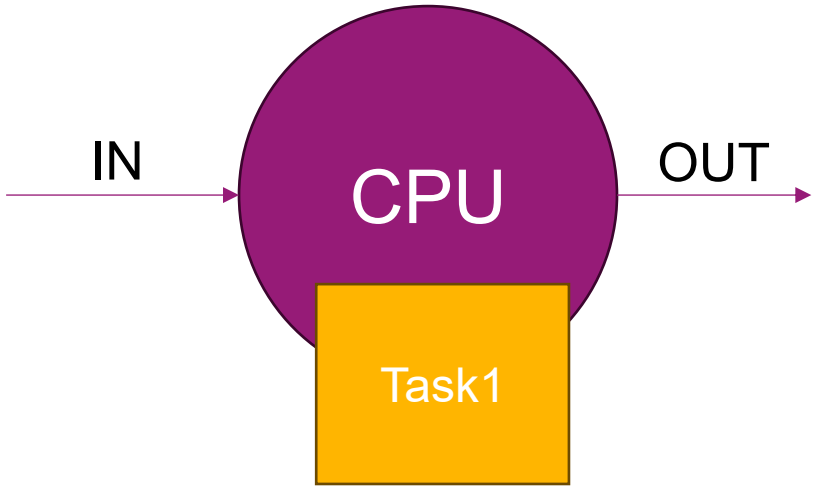
1) SCHEDULING TASKS

- If we have **one task** at a time: easy



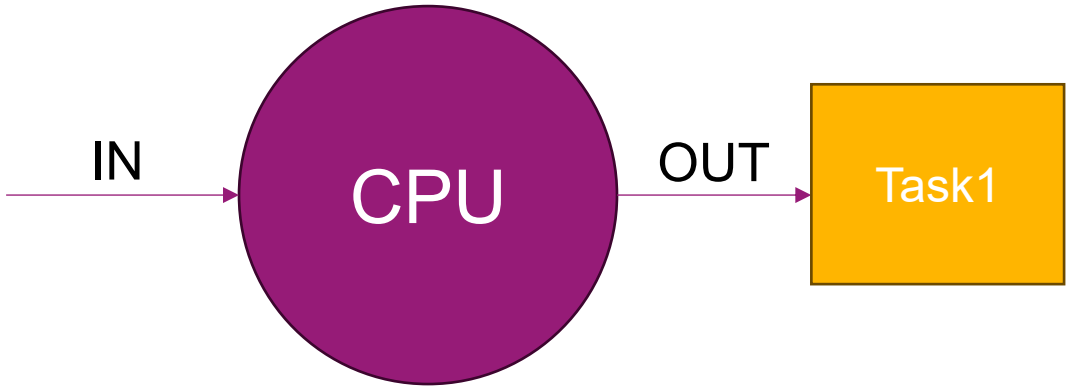
1) SCHEDULING TASKS

- If we have **one task** at a time: easy



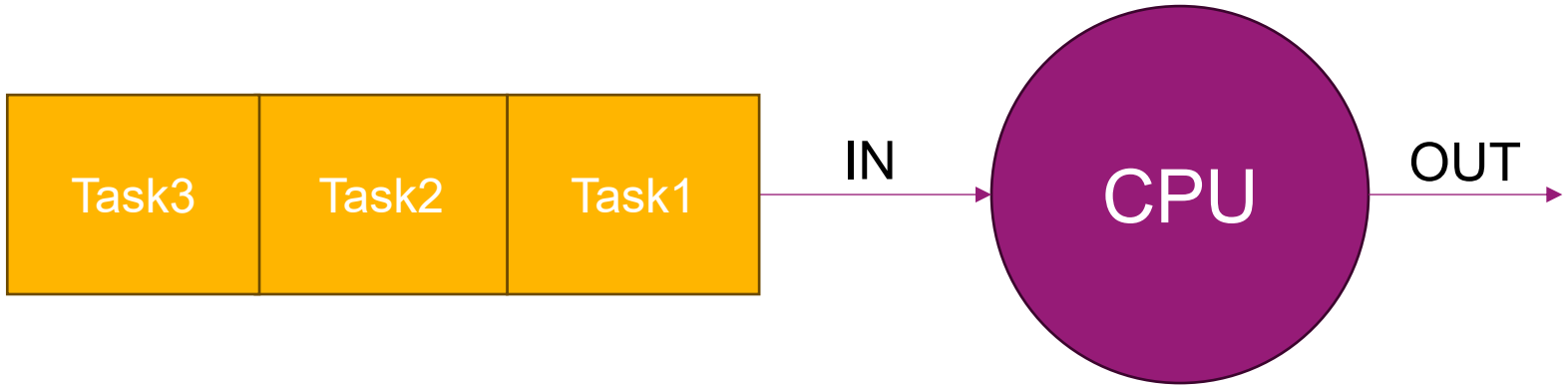
1) SCHEDULING TASKS

- If we have **one task** at a time: easy



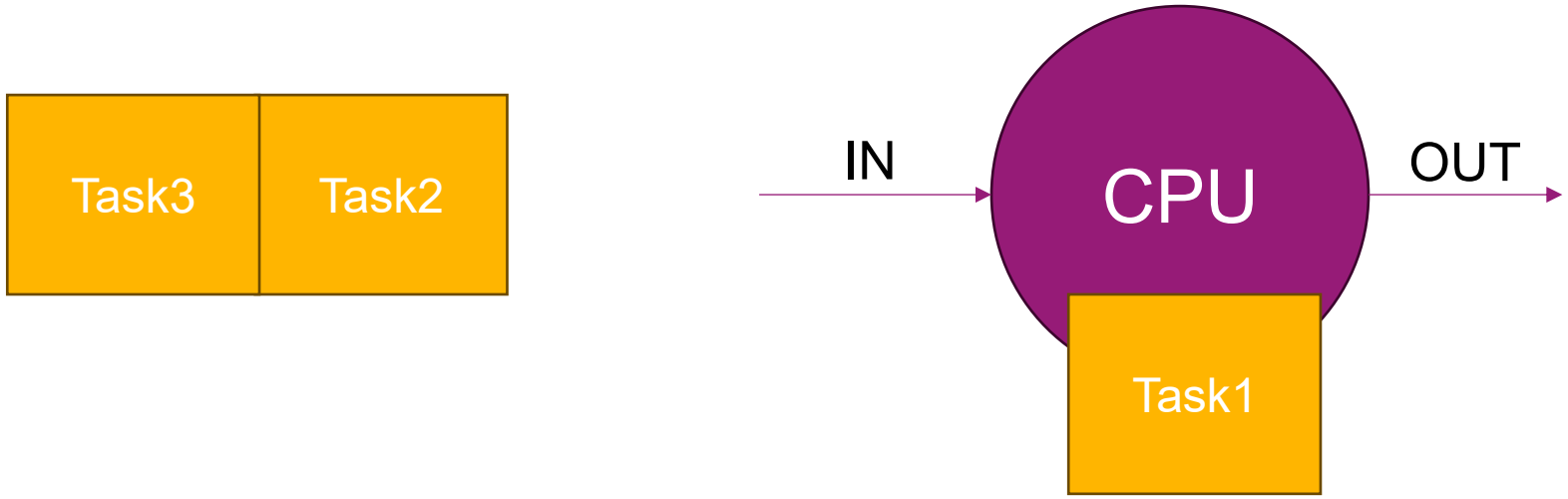
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



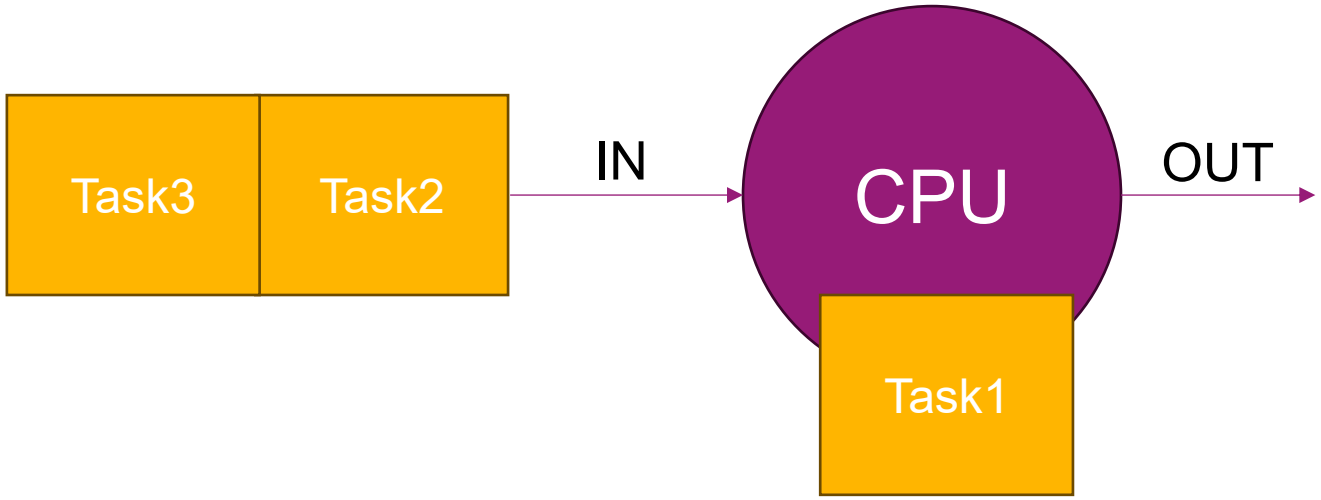
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



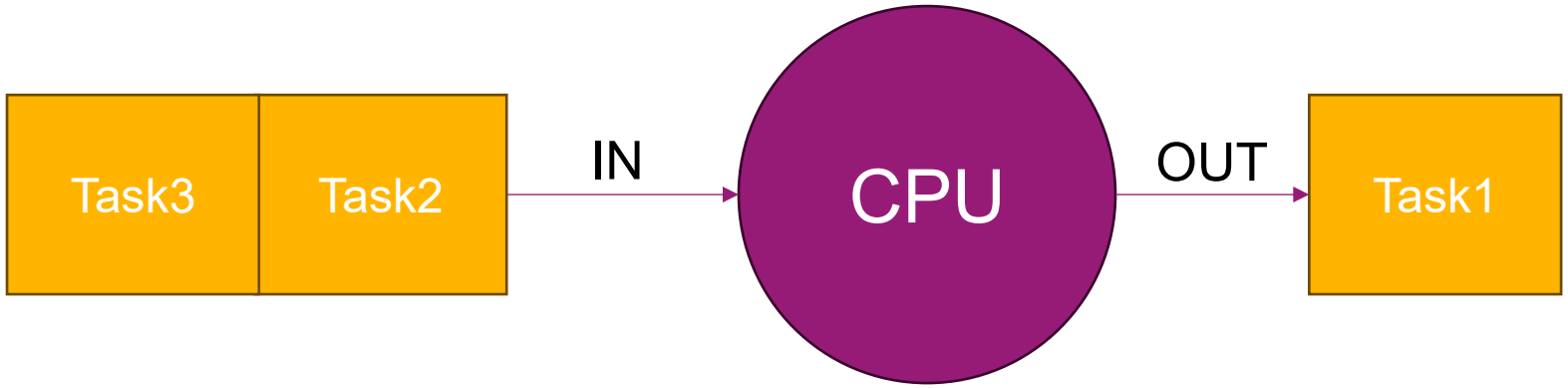
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



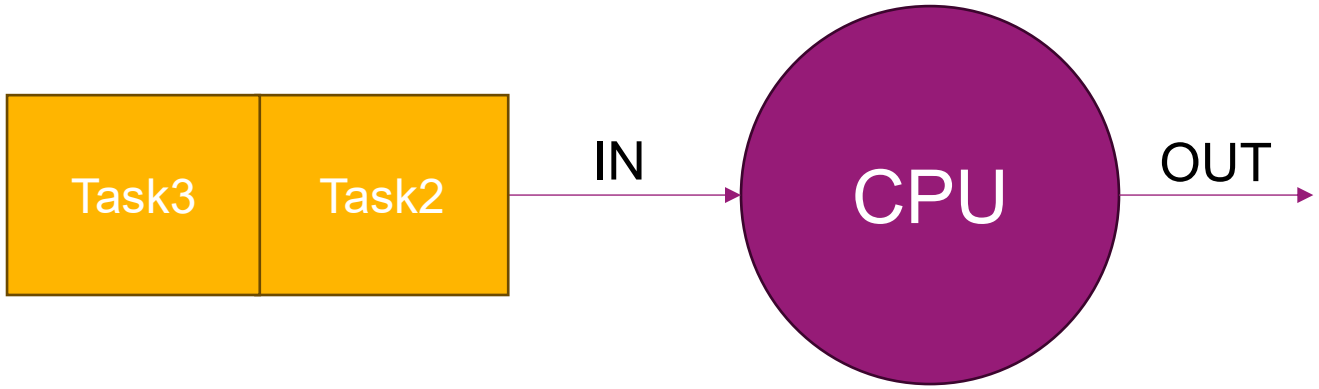
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



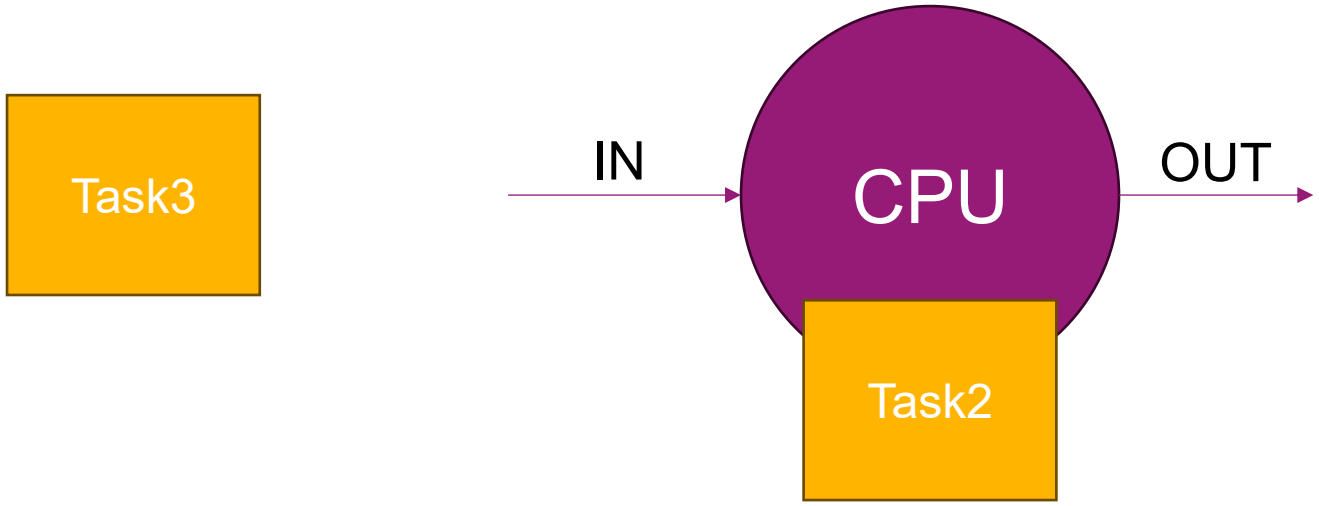
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



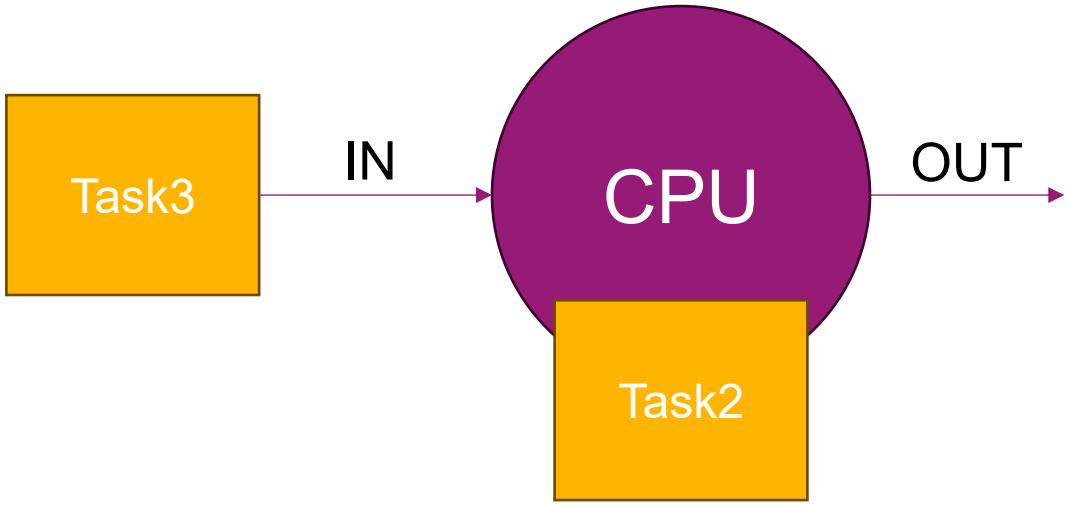
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



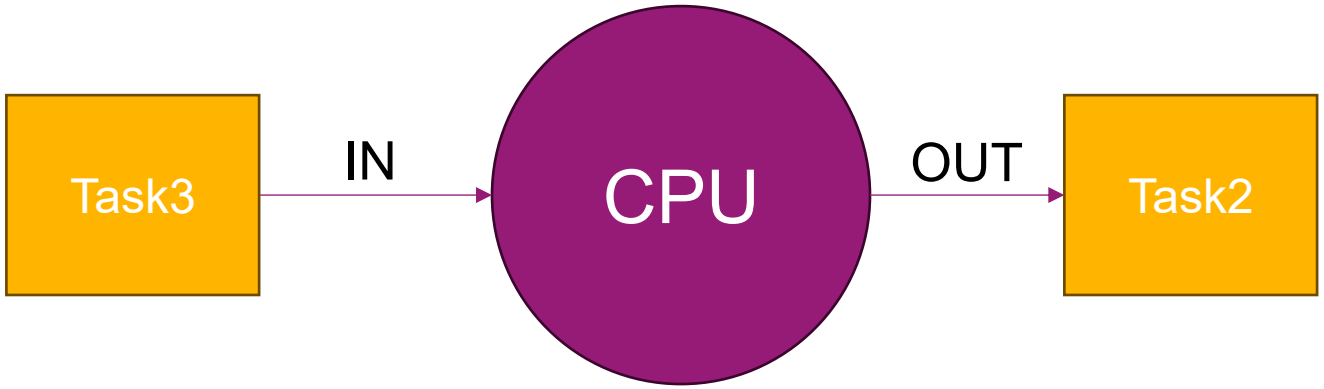
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



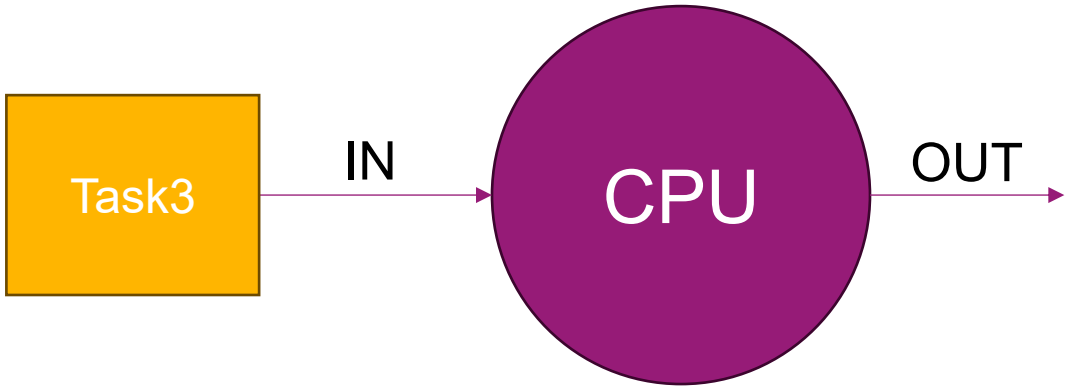
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



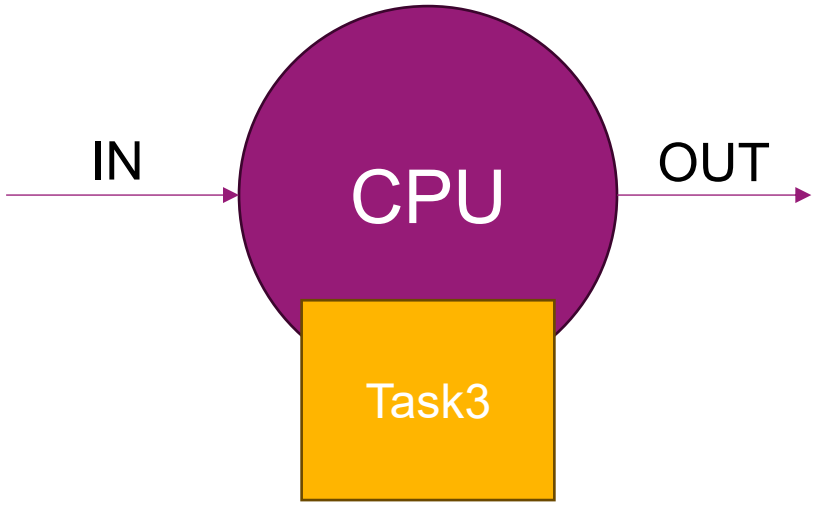
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



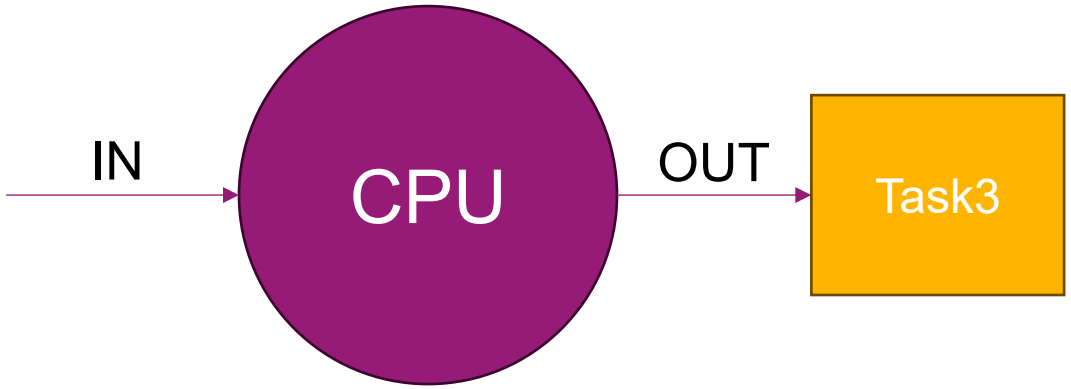
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



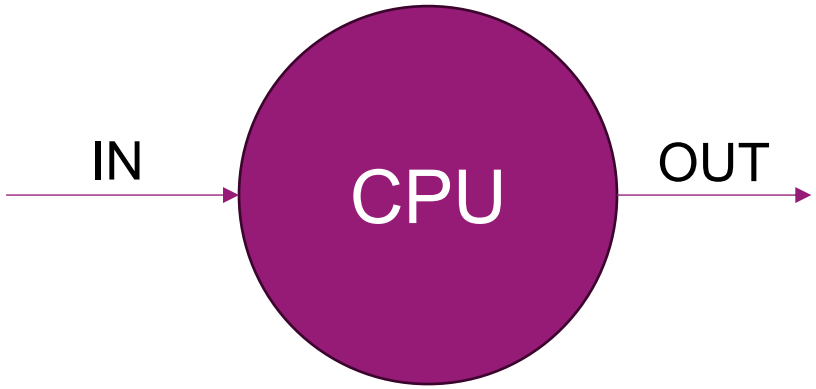
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **can** wait until the end of a task:



1) SCHEDULING TASKS

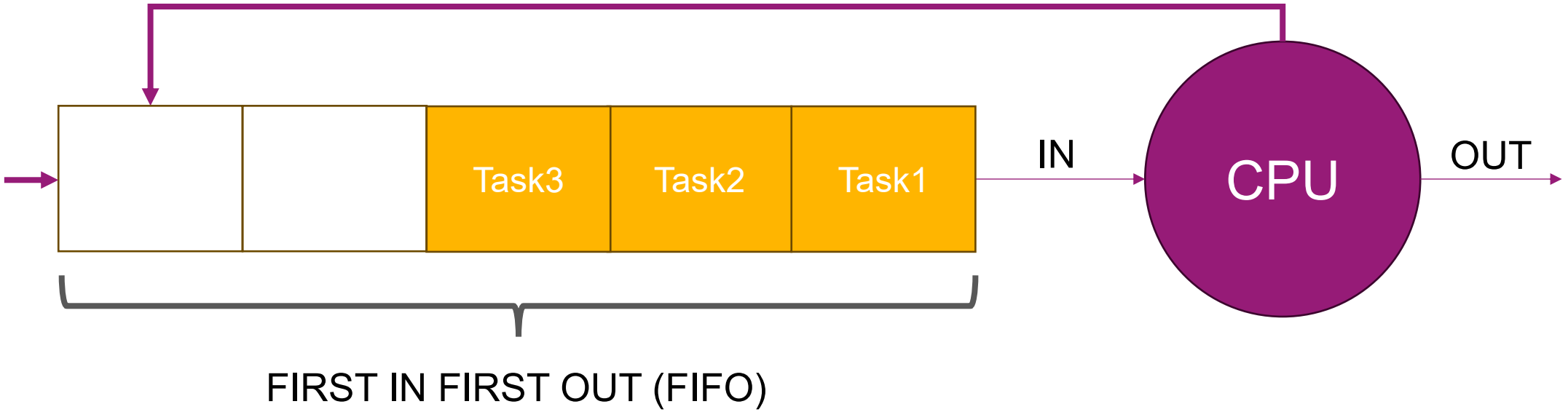
- If we have **many tasks** at a time but we **can** wait until the end of a task:



No more tasks,
the processor enters
the **IDLE** state

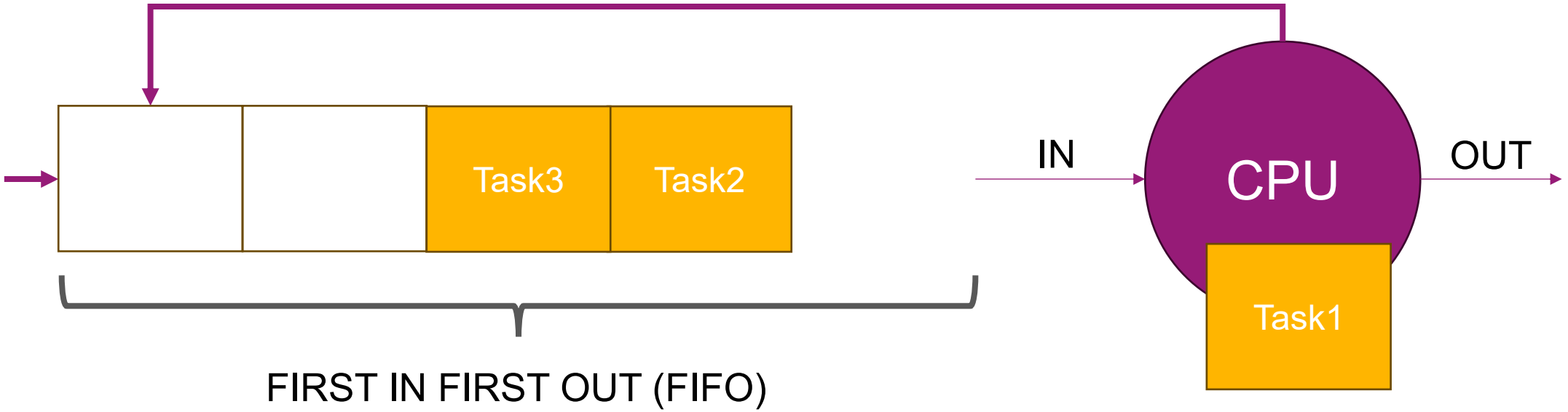
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



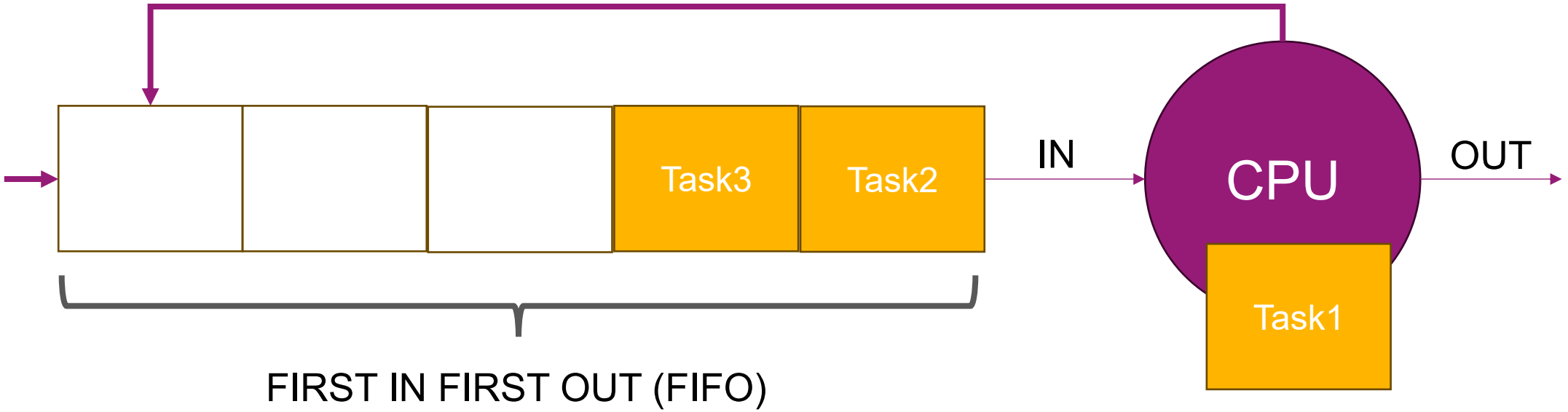
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



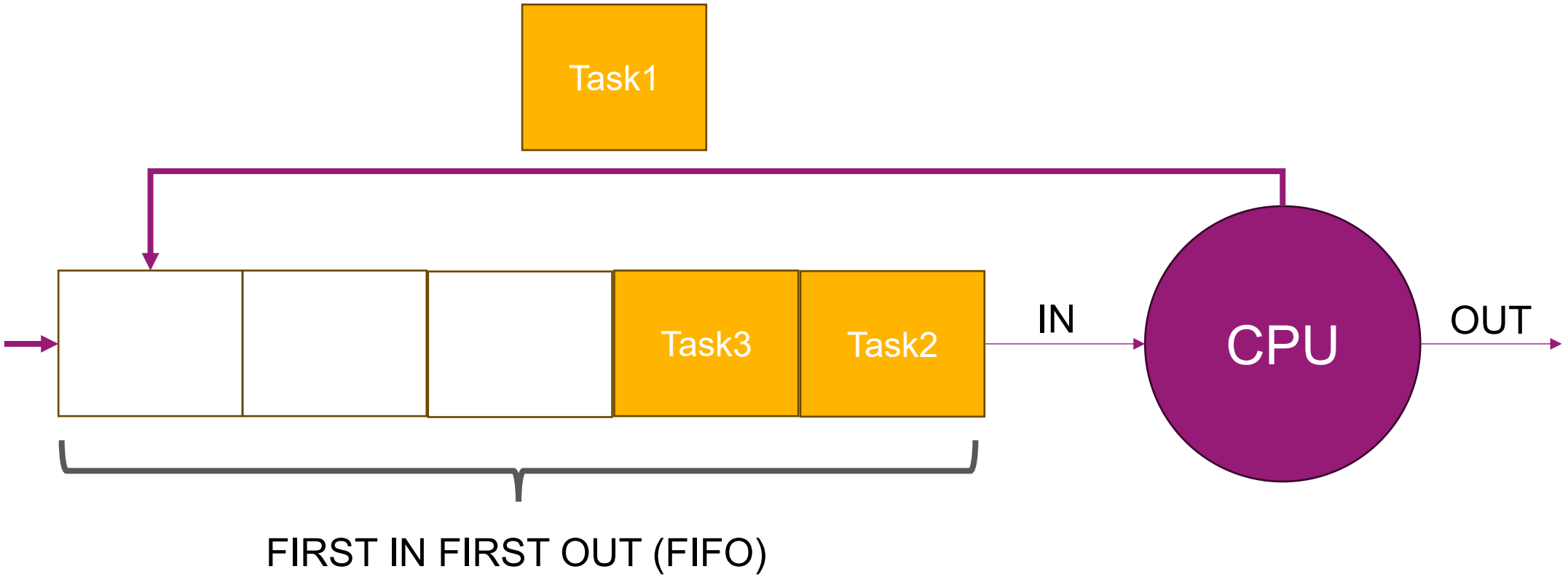
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



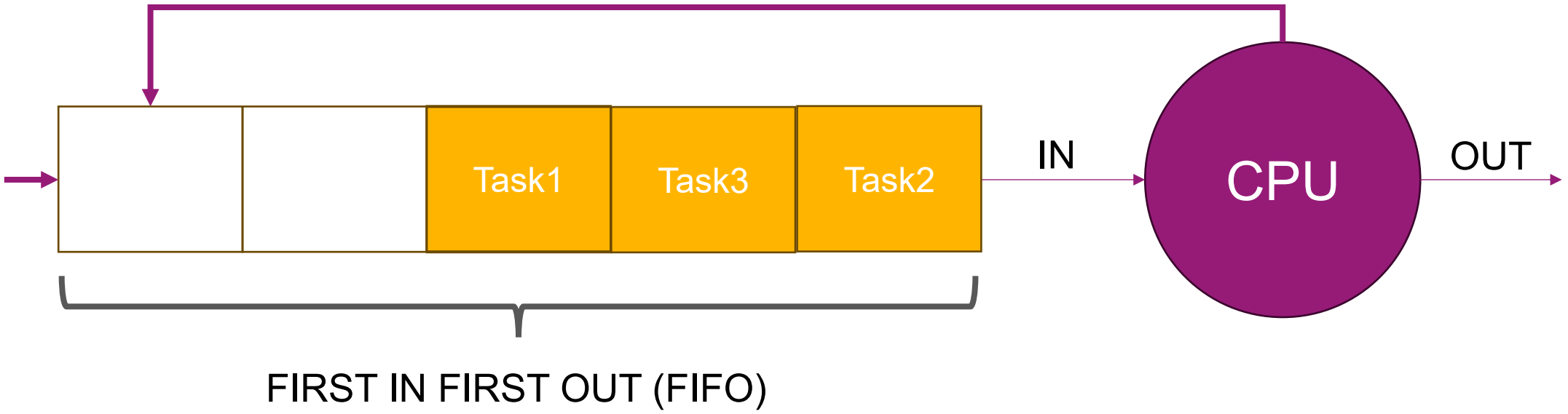
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



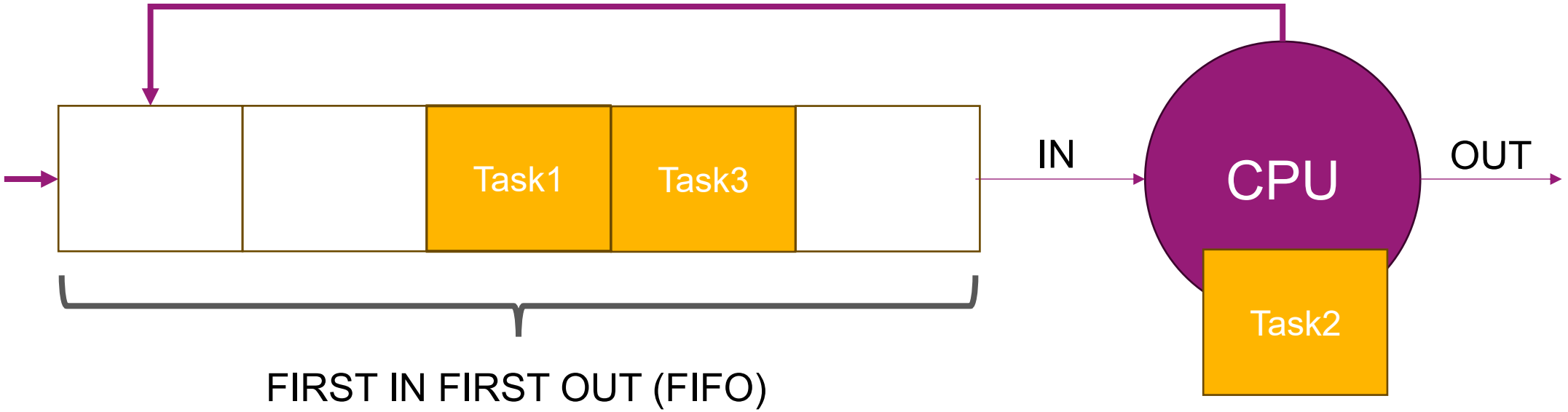
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



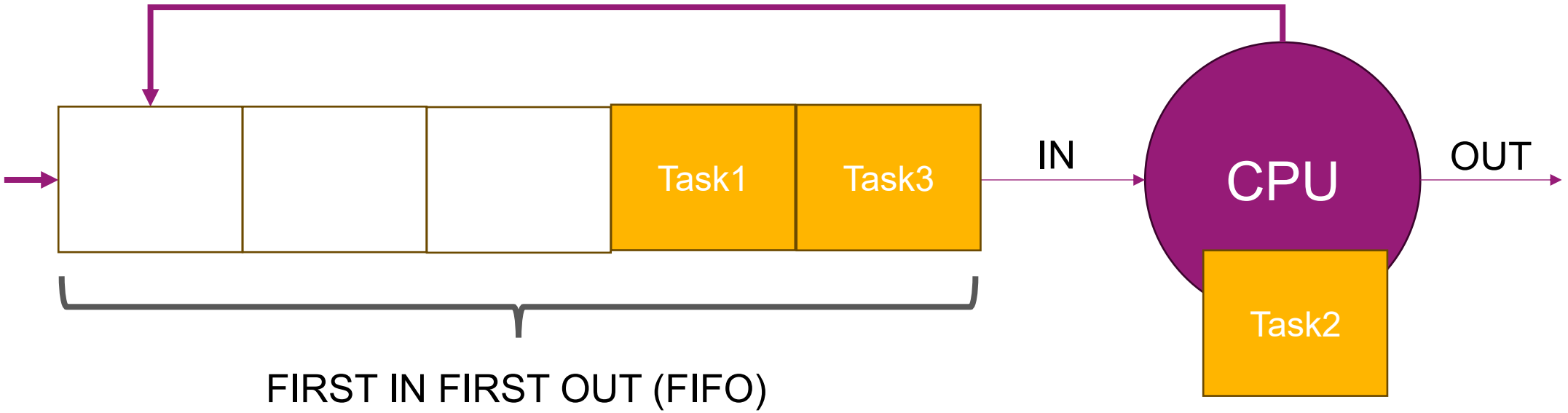
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



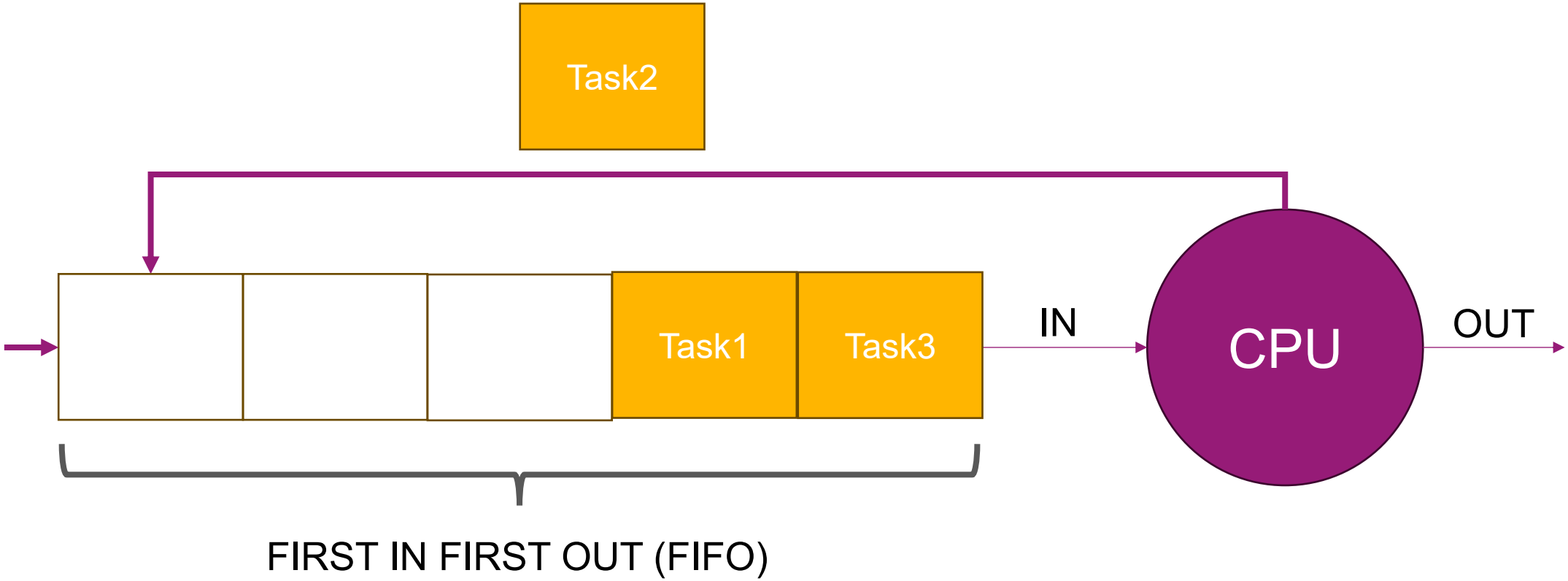
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



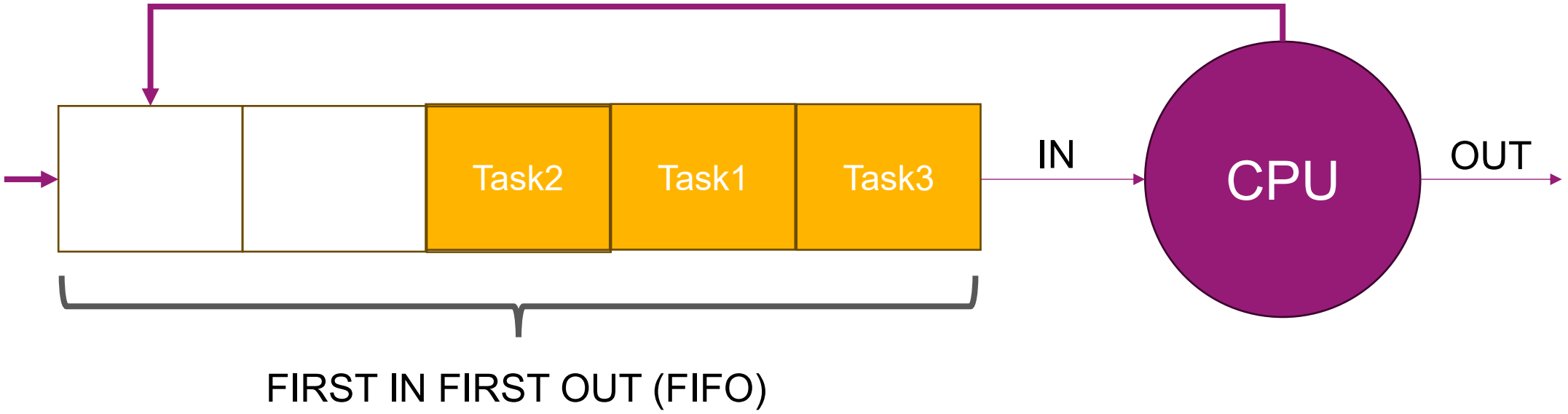
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



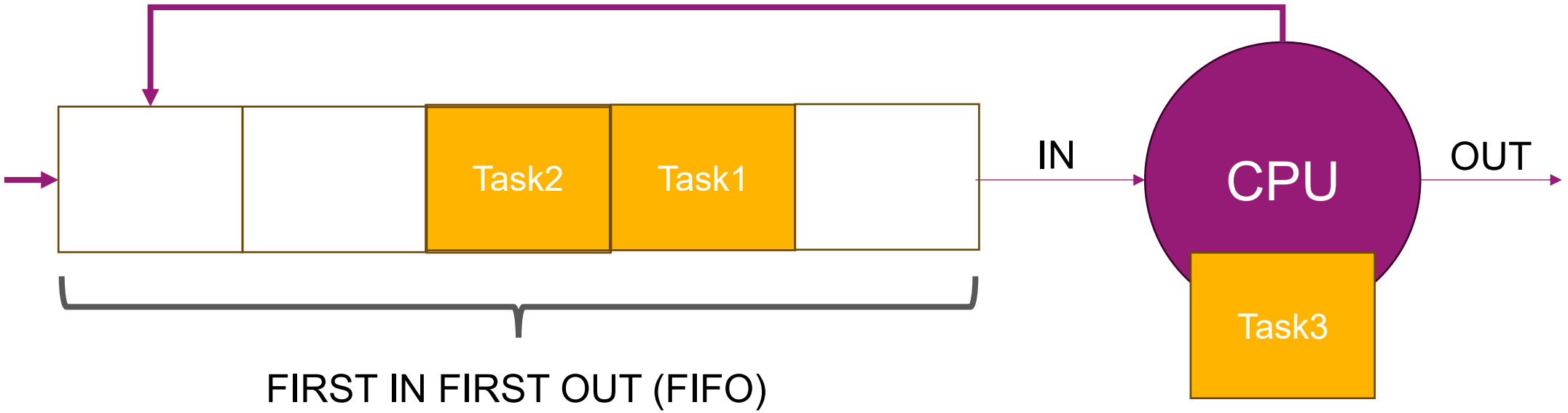
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



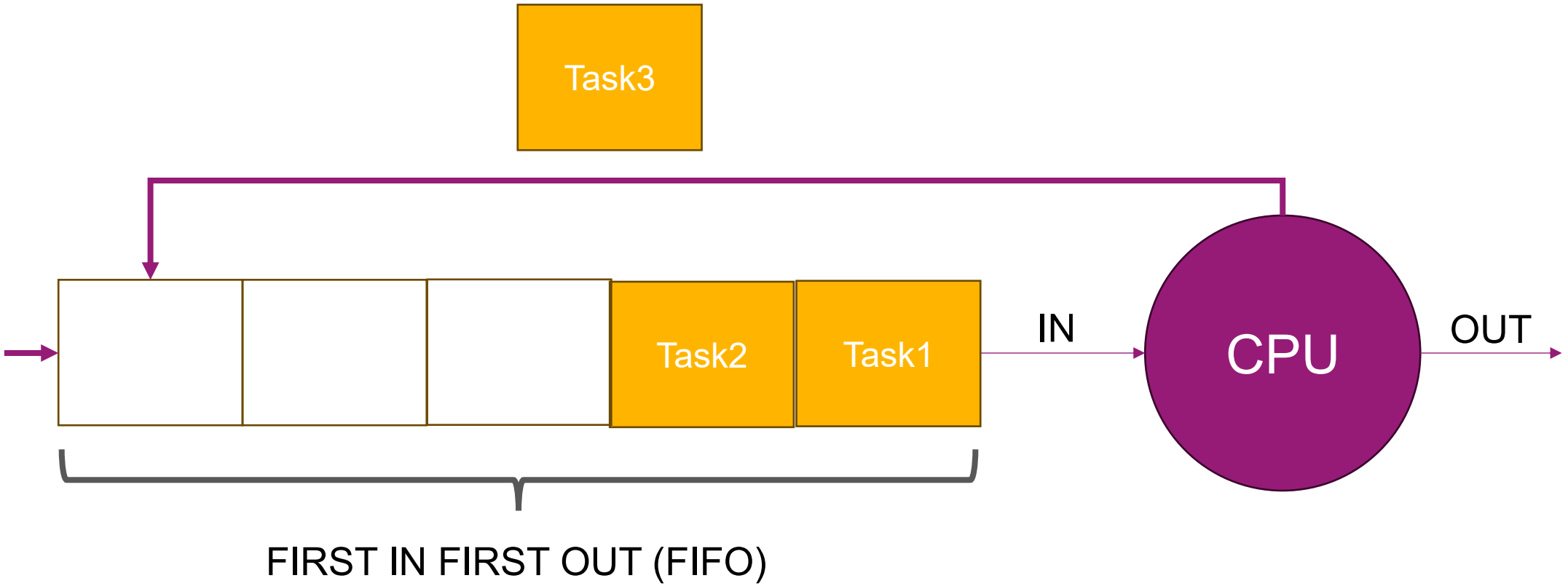
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



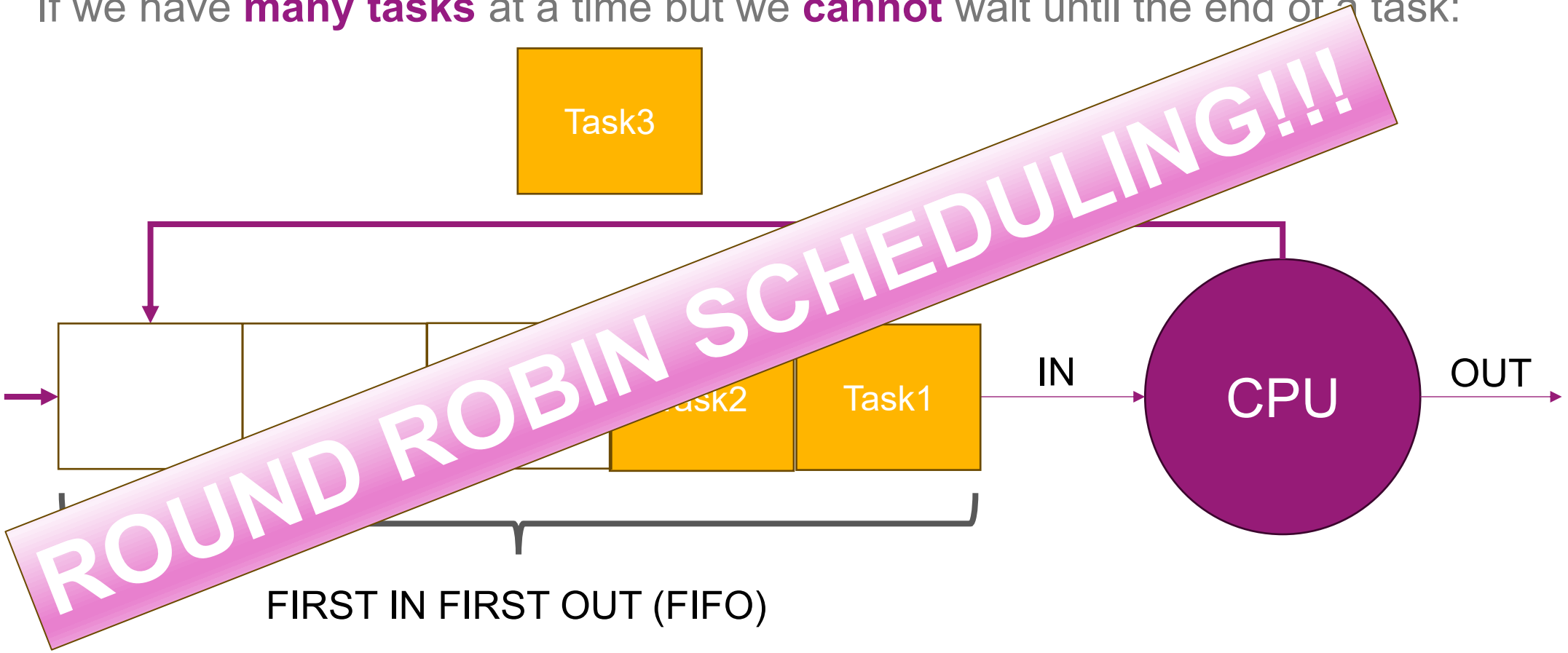
1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



1) SCHEDULING TASKS

- If we have **many tasks** at a time but we **cannot** wait until the end of a task:



1) SCHEDULING TASKS

- The scheduling algorithms use the **PCB** (**Process Control Block**) and **TCB** (**Thread Control Block**) in the different queues
- The operating system manages the PCBs and TCBs during all the life cycle of the **task** (process or thread)

1) SCHEDULING TASKS

Source: https://chat.mistral.ai

1. PCB (Process Control Block)

- **Definition:** In operating systems, a **Process Control Block (PCB)** is a data structure used to store all the information about a process. This includes the process state, program counter, CPU registers, scheduling information, memory management info, I/O status, and accounting data.
- **In Linux:** The closest equivalent to a PCB in Linux is the `task_struct` structure. This structure is defined in the kernel and contains all the information needed to manage a process (or thread, in Linux's implementation).

2. TCB (Thread Control Block)

- **Definition:** A **Thread Control Block (TCB)** is similar to a PCB, but specifically for threads. It stores thread-specific information such as thread ID, program counter, register set, stack pointer, and scheduling information.
- **In Linux:** Linux does **not** distinguish between processes and threads in the traditional sense. Instead, Linux uses the same `task_struct` for both processes and threads. Threads are just processes that share certain resources (like memory space) with their parent process. So, in Linux, there is no separate TCB; the `task_struct` serves both roles.

1) SCHEDULING TASKS

Source: https://chat.mistral.ai

3. The `task_struct` in Linux

- **Role:** The `task_struct` is the central data structure for process/thread management in Linux. It contains:
  - Process ID (PID) and thread group ID (TGID)
  - Process state (running, sleeping, stopped, etc.)
  - Scheduling information (priority, CPU time, etc.)
  - Memory management info (pointers to memory maps, etc.)
  - File system info (open files, working directory, etc.)
  - Signal handling info
  - And much more
- **Location:** The `task_struct` is defined in the kernel source, typically in `include/linux/sched.h`.

1) SCHEDULING TASKS

- The task_struct in Linux is defined in the source code of the kernel in ”/include/linux/sched.h”

```
819  struct task_struct {
820  #ifdef CONFIG_THREAD_INFO_IN_TASK
821      /*
822       * For reasons of header soup (see current_thread_info()), this
823       * must be the first element of task_struct.
824       */
825      struct thread_info      thread_info;
826  #endif
827      unsigned int            __state;
828
829      /* saved state for "spinlock sleepers" */
830      unsigned int            saved_state;
831
832      /*
833       * This begins the randomizable portion of task_struct. Only
834       * scheduling-critical items should be added above here.
835       */
836      randomized_struct_fields_start
837
838      void                    *stack;
839      refcount_t              usage;
840      /* Per task flags (PF_*), defined further below: */
841      unsigned int                flags;
842      unsigned int                ptrace;
843
```

Source: https://elixir.bootlin.com/linux/v6.17.8/source/include/linux/sched.h

1) SCHEDULING TASKS

- The task_struct in Linux is defined in the source code of the kernel in ”/include/linux/sched.h”

```
854     /*
855      * recent_used_cpu is initially set as the last CPU used by a task
856      * that wakes affine another task. Waker/wakee relationships can
857      * push tasks around a CPU where each wakeup moves to the next one.
858      * Tracking a recently used CPU allows a quick search for a recently
859      * used CPU that may be idle.
860      */
861     int             recent_used_cpu;
862     int             wake_cpu;
863     int             on_rq;
864
865     int             prio;
866     int             static_prio;
867     int             normal_prio;
868     unsigned int            rt_priority;
869
870     struct sched_entity     se;
871     struct sched_rt_entity      rt;
872     struct sched_dl_entity      dl;
873     struct sched_dl_entity      *dl_server;
874 #ifdef CONFIG_SCHED_CLASS_EXT
875     struct sched_ext_entity     scx;
876 #endif
877     const struct sched_class    *sched_class;
```

Source: https://elixir.bootlin.com/linux/v6.17.8/source/include/linux/sched.h

Field/Group	Description
Process State	volatile long state: Indicates if the task is runnable, sleeping, stopped, etc.
Process ID	pid_t pid: Unique process ID. For threads, this is the thread ID.
Thread Group ID	pid_t tgid: Identifies the thread group (process) to which the thread belongs.
Parent/Child Relations	Pointers to parent, children, and siblings (*parent, *children, *sibling).
Memory Management	*mm: Points to the memory descriptor (mm_struct), shared by all threads in a process.
Scheduling Info	unsigned int policy, int prio, unsigned int rt_priority: Scheduling policy and priority.
CPU Time Accounting	unsigned long utime, stime: User and system CPU time used.
Credentials	uid_t uid, gid_t gid: User and group IDs.
File System Info	*fs: Filesystem information, including working directory and root.
Signal Handling	struct signal_struct *signal: Manages pending and blocked signals.
Timers	struct timer_list real_timer: Used for process timers (e.g., alarm()).
Process Tree Navigation	*group_leader: Points to the main thread (process leader) of the thread group.
CPU Affinity	cpumask_t cpus_allowed: CPUs on which the task may run.
Context Switch Counts	unsigned long nvcsw, nivcsw: Voluntary and involuntary context switch counts.

Source: https://chat.mistral.ai

2) TYPES OF SCHEDULERS

- There are two types of schedulers:
  - **Preemptive** schedulers
  - **Cooperative/Non-preemptive** schedulers

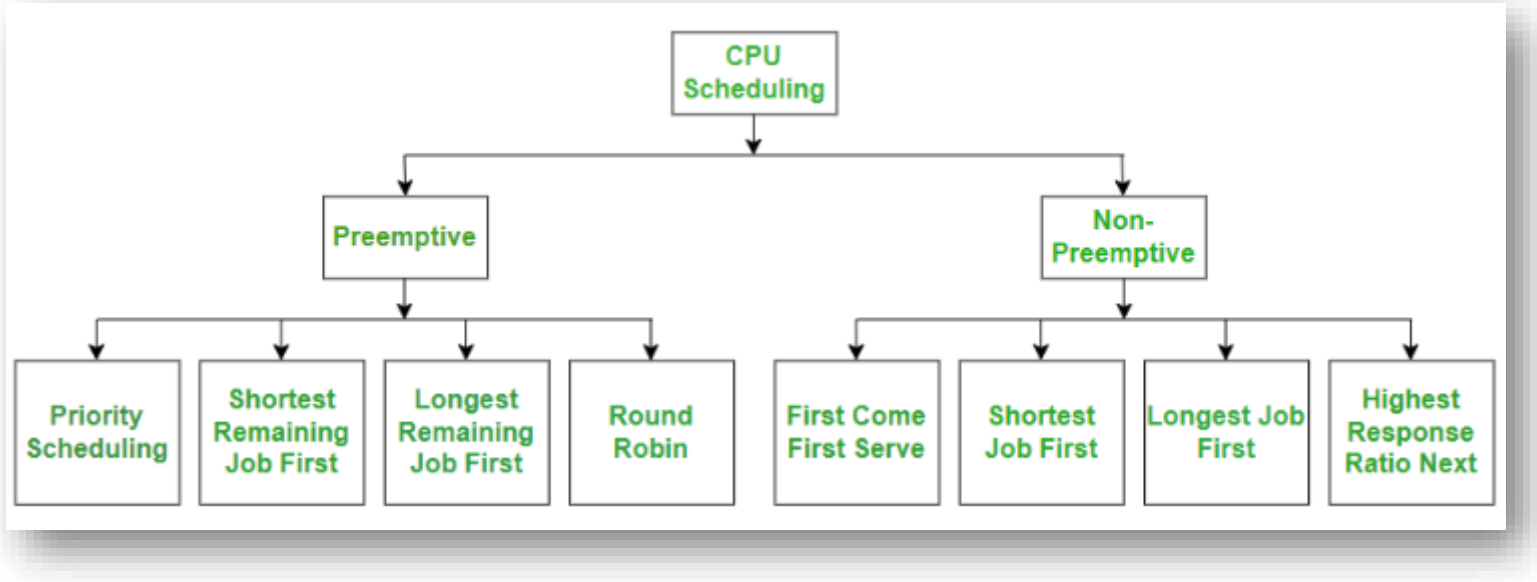
  In **preemptive multitasking**, the operating system can initiate a context switching from the running process to another process. In other words, the operating system allows stopping the execution of the currently running process and allocating the CPU to some other process. The OS uses some criteria to decide for how long a process should execute before allowing another process to use the operating system. The mechanism of taking control of the operating system from one process and giving it to another process is called preempting or preemption.

  In **cooperative multitasking**, the operating system never initiates context switching from the running process to another process. A context switch occurs only when the processes voluntarily yield control periodically or when idle or logically blocked to allow multiple applications to execute simultaneously. Also, in this multitasking, all the processes cooperate for the scheduling scheme to work.

Source: (geeksforgeeks, https://www.geeksforgeeks.org/difference-between-preemptive-and-cooperative-multitasking/, 2024)

2) TYPES OF SCHEDULERS

- There are two types of schedulers:
  - **Preemptive** schedulers
  - **Cooperative/Non-preemptive** schedulers



Source: (geeksforgeeks, https://www.geeksforgeeks.org/cpu-scheduling-in-operating-systems/, 2024)

3) PREEMPTIVE SCHEDULING

- The Round Robin is a **preemptive** scheduling method that allows the CPU to be shared in between different processes/threads in a waiting queue (FIFO)

- It creates a super infinite loop that takes every process/thread one at a time for a period of time, the **time quantum** Δt, and run it on the CPU until Δt is over or an exception occurs

- The time quantum is also called **time slice** or **time unit**.

3) PREEMPTIVE SCHEDULING

- In operating systems, a time quantum (also known as a time slice or time unit) is a fixed interval of time allocated to a process for execution before being preempted by the scheduler. This allows for time-sharing among multiple processes, creating the illusion of parallel processing.
- Modern operating systems, such as Windows, use time slices ranging from 20 milliseconds to 120 milliseconds, depending on the version and foreground/background program status.
- In Linux, the time quantum is typically very small, on the order of milliseconds, and is managed by a weighted fair queueing (WFQ) implementation.

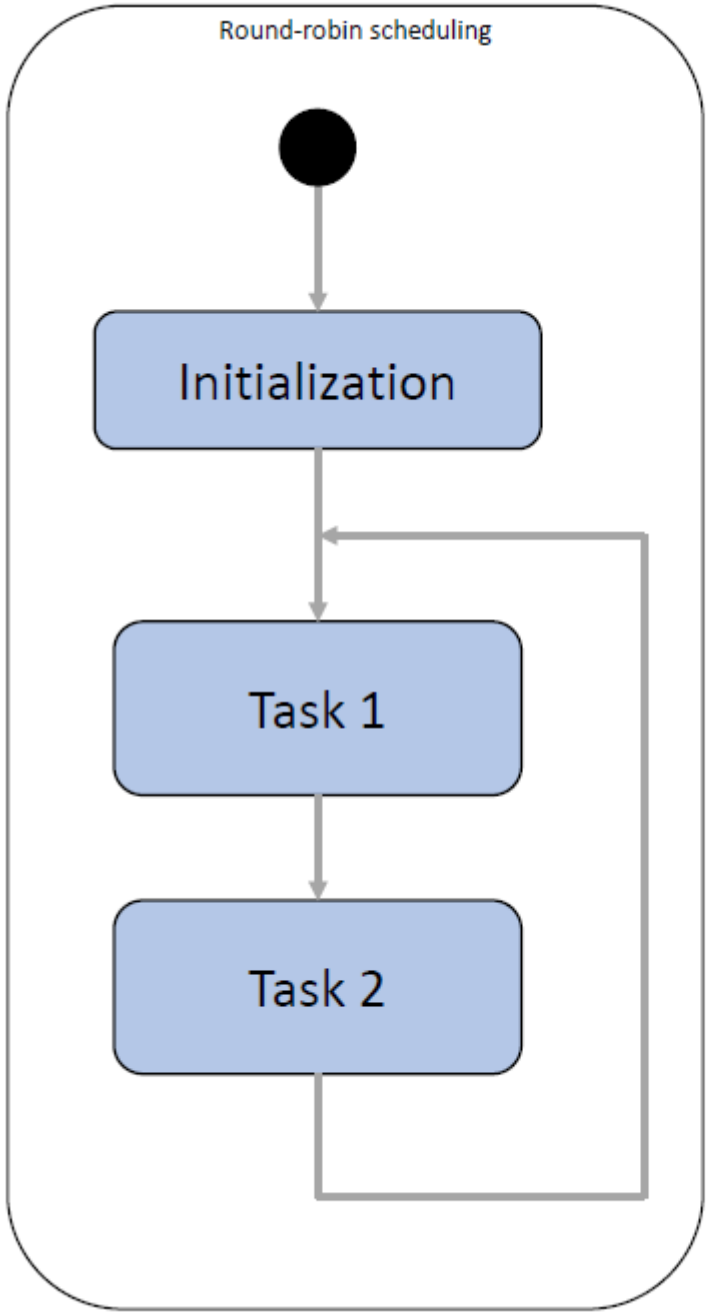
Source: Brave Browser AI

3) PREEMPTIVE SCHEDULING

- Preemption occurs when a process's time slice expires, triggering an interrupt that allows the operating system kernel to switch to the next process. This ensures that the processor's time is shared among multiple tasks, creating the illusion of parallel processing.

- In summary, the quantum of time in operating systems is a fixed interval of time allocated to a process for execution, ranging from a few milliseconds to a few hundred milliseconds, depending on the operating system and scheduling algorithm used.

Source: Brave Browser AI

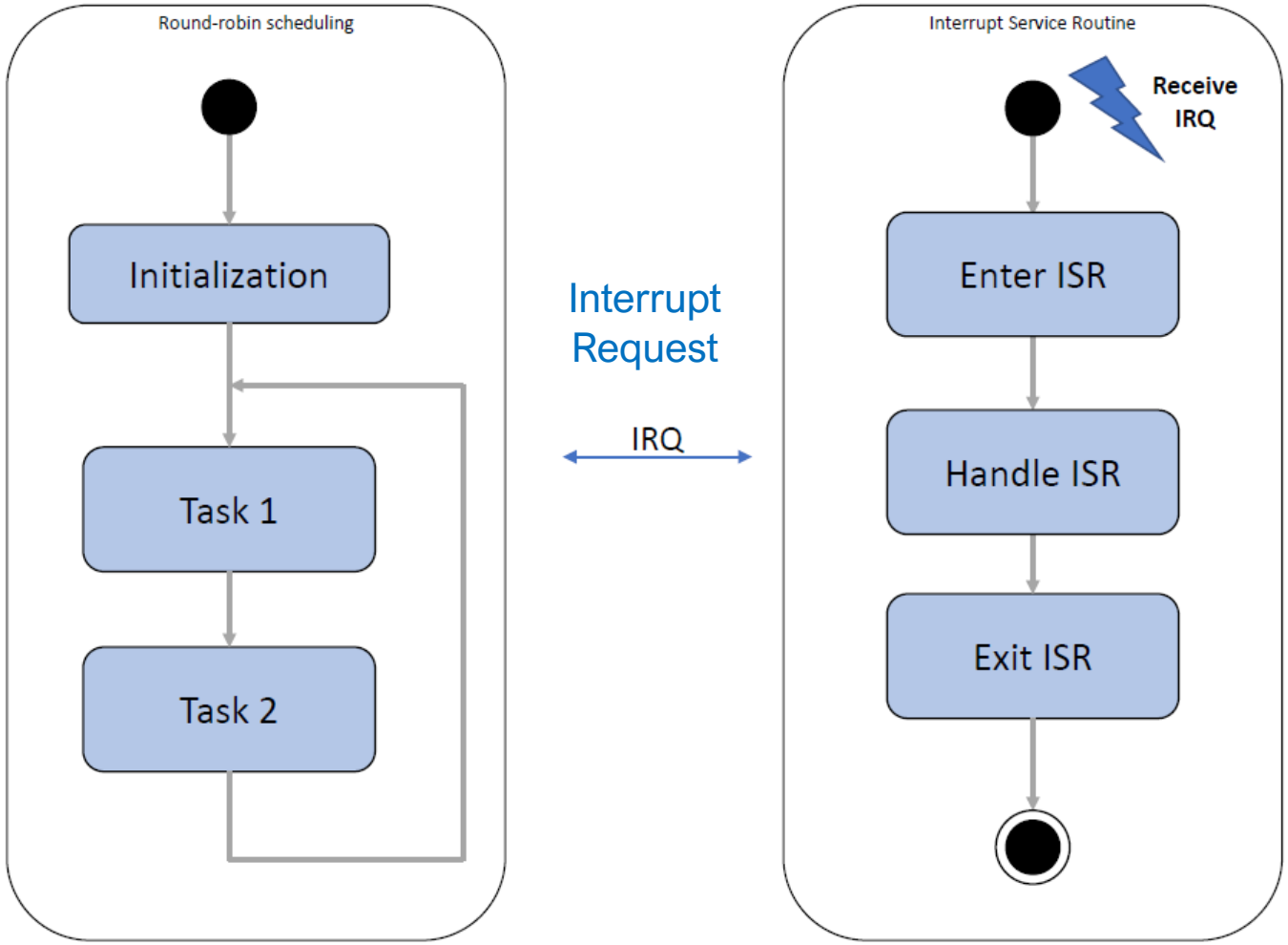



```c
int main()
{
    System_Init();

    // Begin round robin scheduling
    while (1)
    {
        Task_1();

        Task_2();
    }

    return 0;
}
```

4) INTERRUPTS

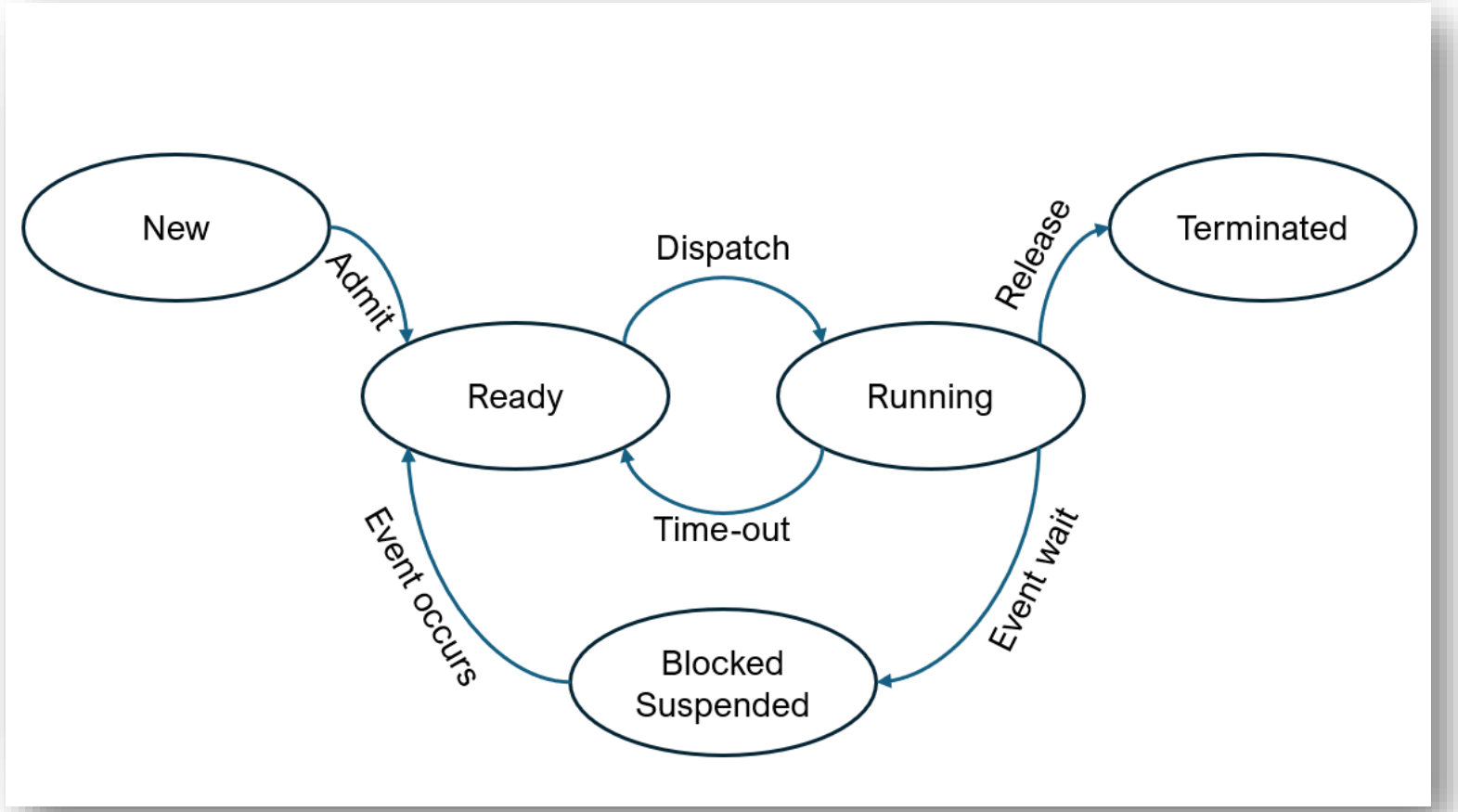
- The Round Robin can (and must) be interrupted when an event occurs on the system
- An interruption can occur at any moment
- The operating system must do something with this event
- An interrupt can be:
  - Software: a computing/process is terminated…
  - Hardware: a file as been read, a keyboard as been typed by the user…

4) INTERRUPTS



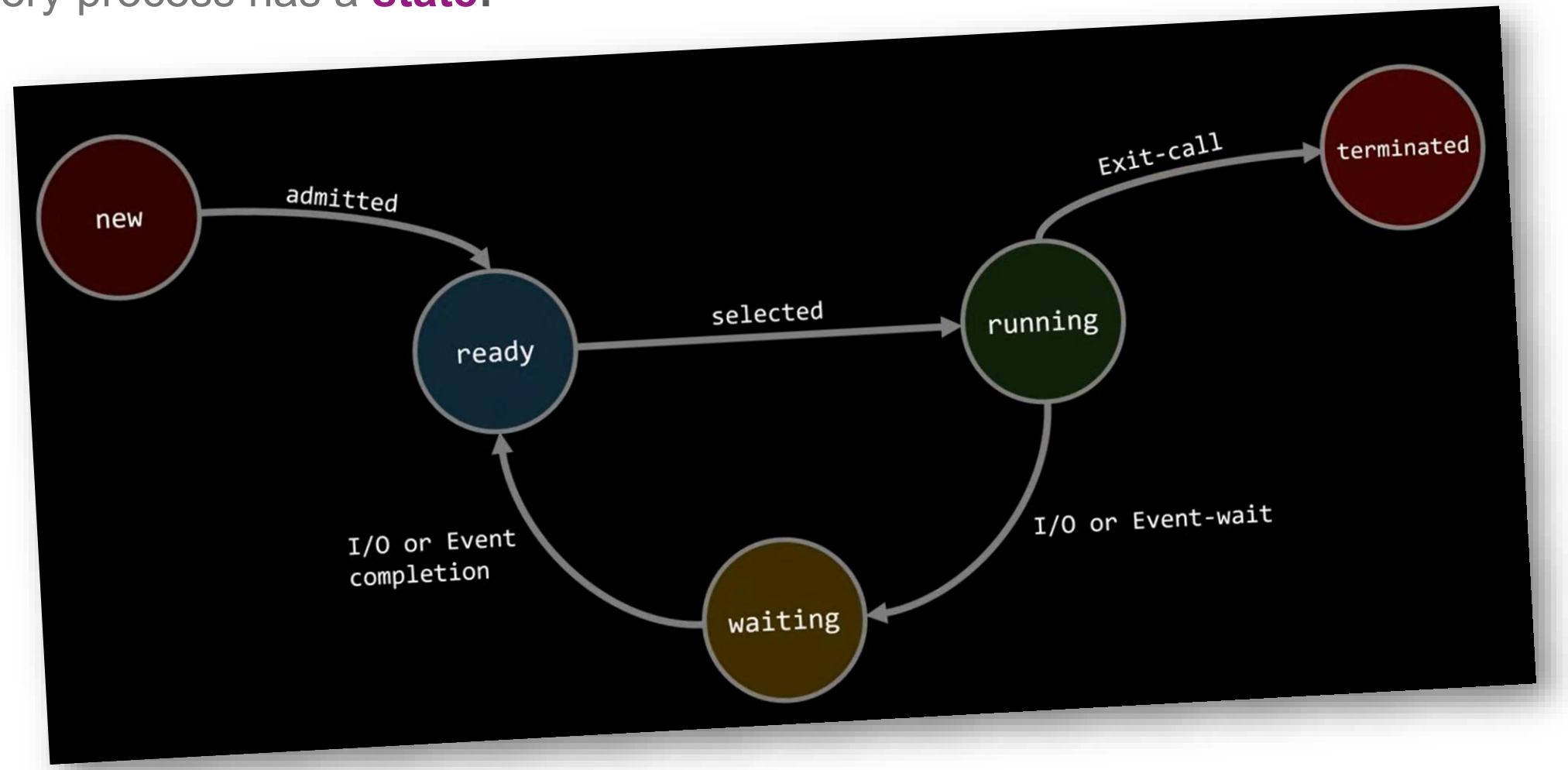
4) INTERRUPTS

- Every process has a **state**:



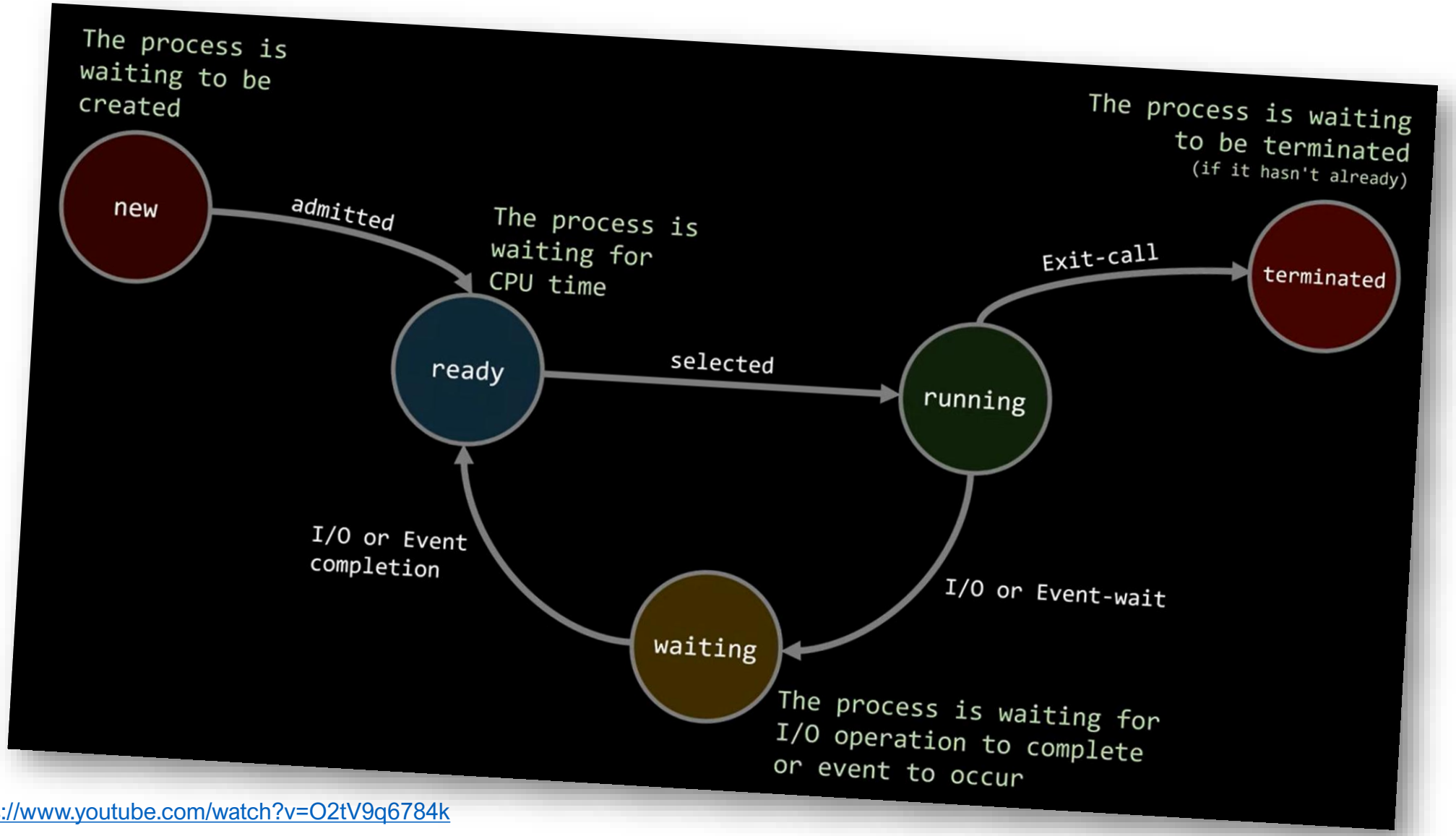
4) INTERRUPTS

- Every process has a **state**:



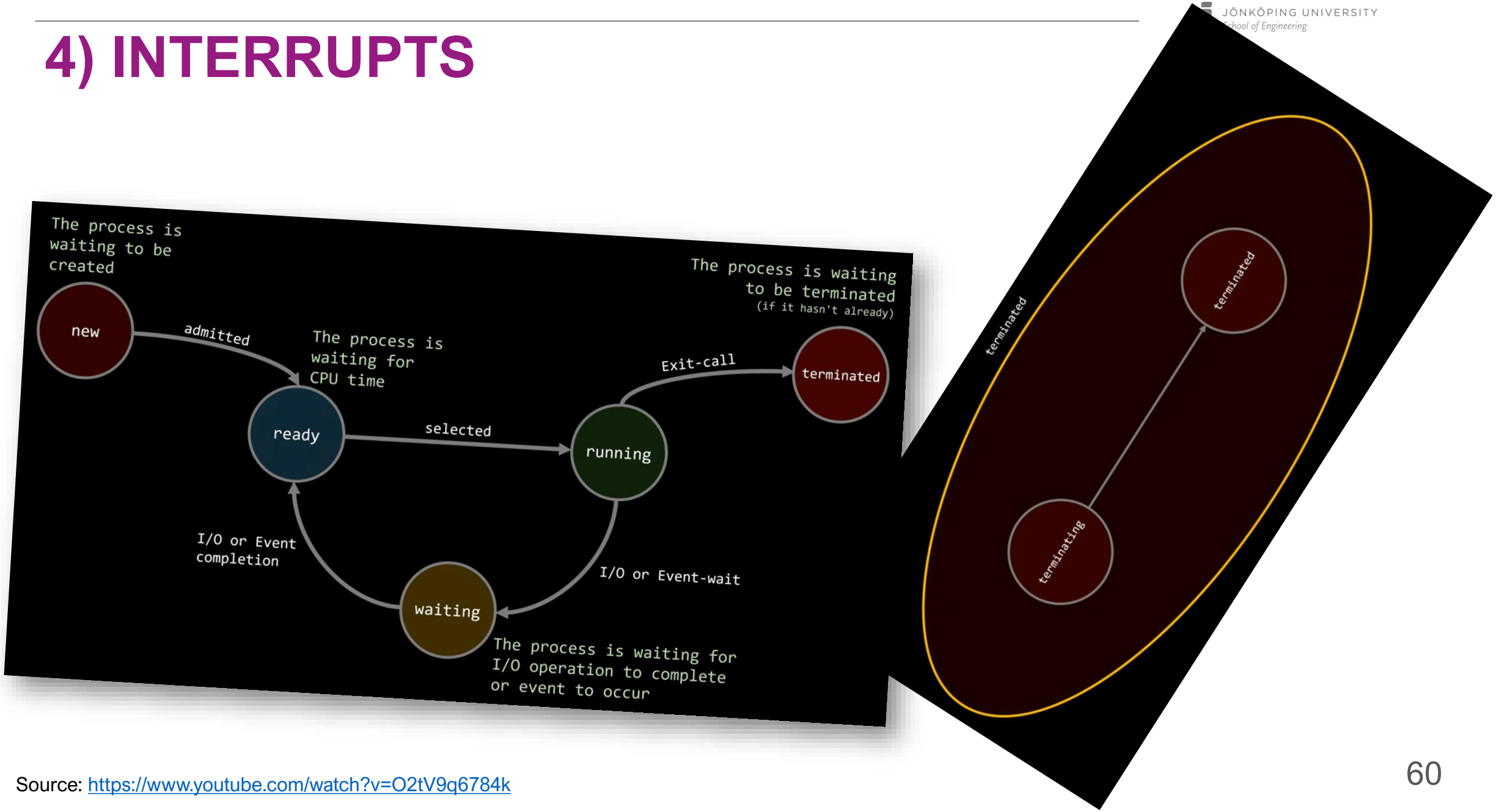
Source: https://www.youtube.com/watch?v=O2tV9q6784k

4) INTERRUPTS



Source: https://www.youtube.com/watch?v=O2tV9q6784k

4) INTERRUPTS



Source: https://www.youtube.com/watch?v=O2tV9q6784k

- A note about **Zombie processes**
  - Zombie processes arise when child processes finish but remain in the process table due to their parent not calling `wait()` to collect their exit status
  - They are identifiable by a status of "Z" (zombie) or "defunct" in commands like `ps` or `top`.
- Zombies cannot be killed by “kill -9” since they are not running anymore.

- The correct method for removal is to deal with their parent process:
  - Identify the zombie’s PID: ps aux | grep 'Z'
  - Find the parent PID (PPID): ps -o ppid= -p <zombie_PID>
  - Signal the parent to perform cleanup: kill -s SIGCHLD <parent_PID>
  - If signaling does not work, forcefully terminate the parent: kill -9 <parent_PID>

Source: (Mistral AI, https://mistral.ai, 2025)

- **Automating Zombie Cleanup**
  - Bash scripts can automate finding zombie processes, identifying their parents, and signaling or killing the parent to clear zombies

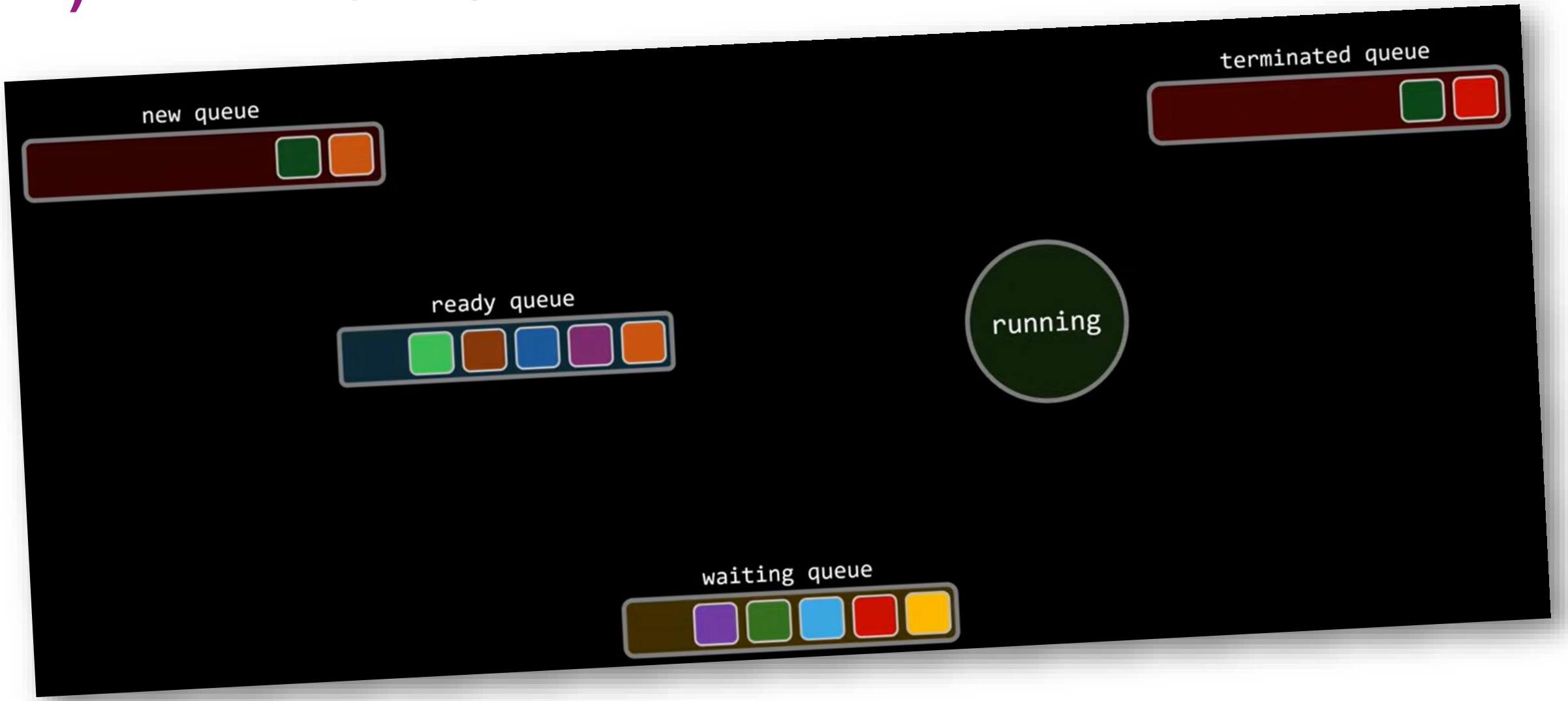
```
#bash
zombie_pids=$(ps aux | grep -w Z | awk '{print $2}’)
for zombie_pid in $zombie_pids; do
    parent_pid=$(ps -o ppid= -p $zombie_pid)
    kill -1 $parent_pid
done
```

- This script sends a `SIGCHLD` to the parent of each detected zombie

- Prevention Tips : proper process handling in applications, ensuring the parent collects child exit statuses with `wait()`, helps prevent zombies

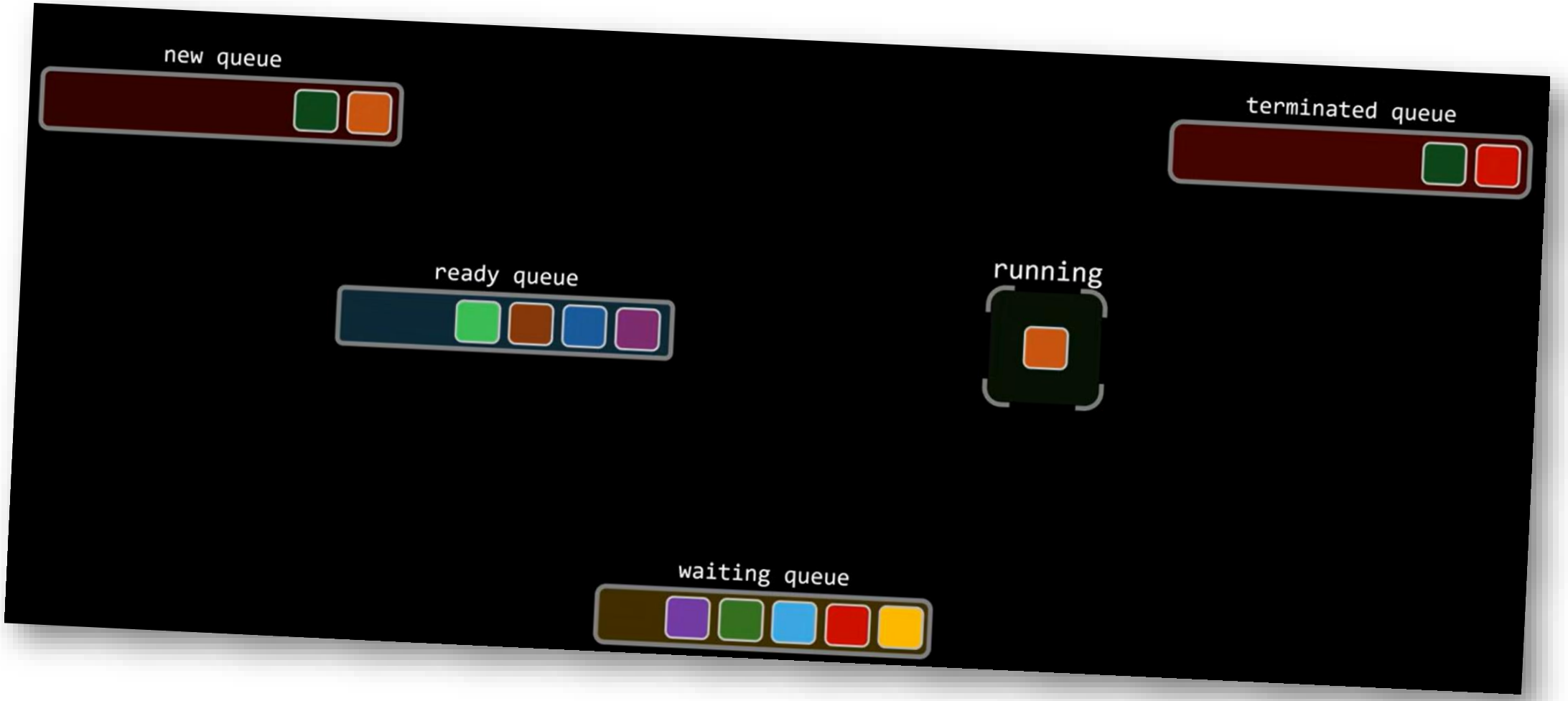
Source: (Mistral AI, https://mistral.ai, 2025)

4) INTERRUPTS



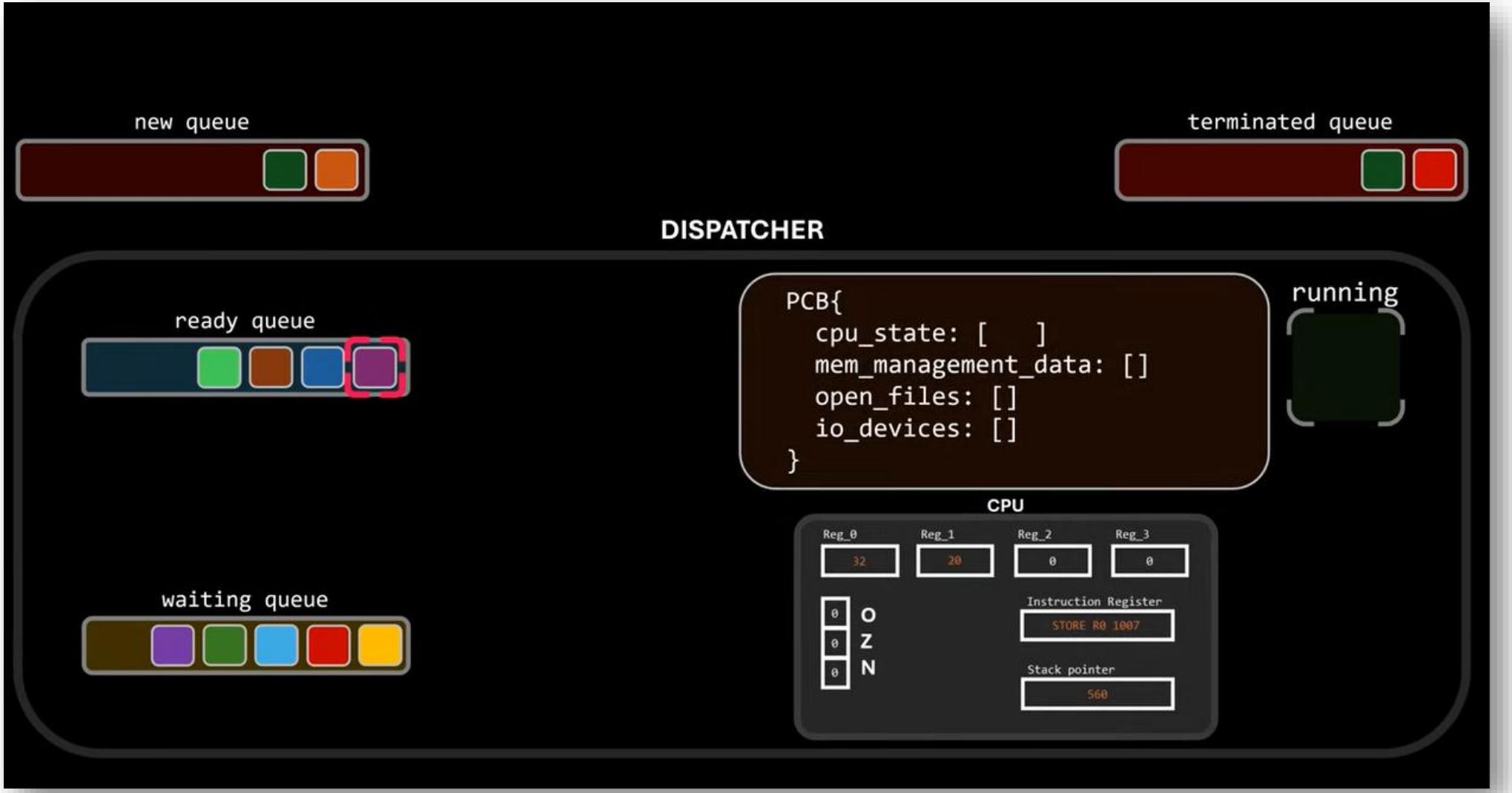
Source: https://www.youtube.com/watch?v=O2tV9q6784k

4) INTERRUPTS



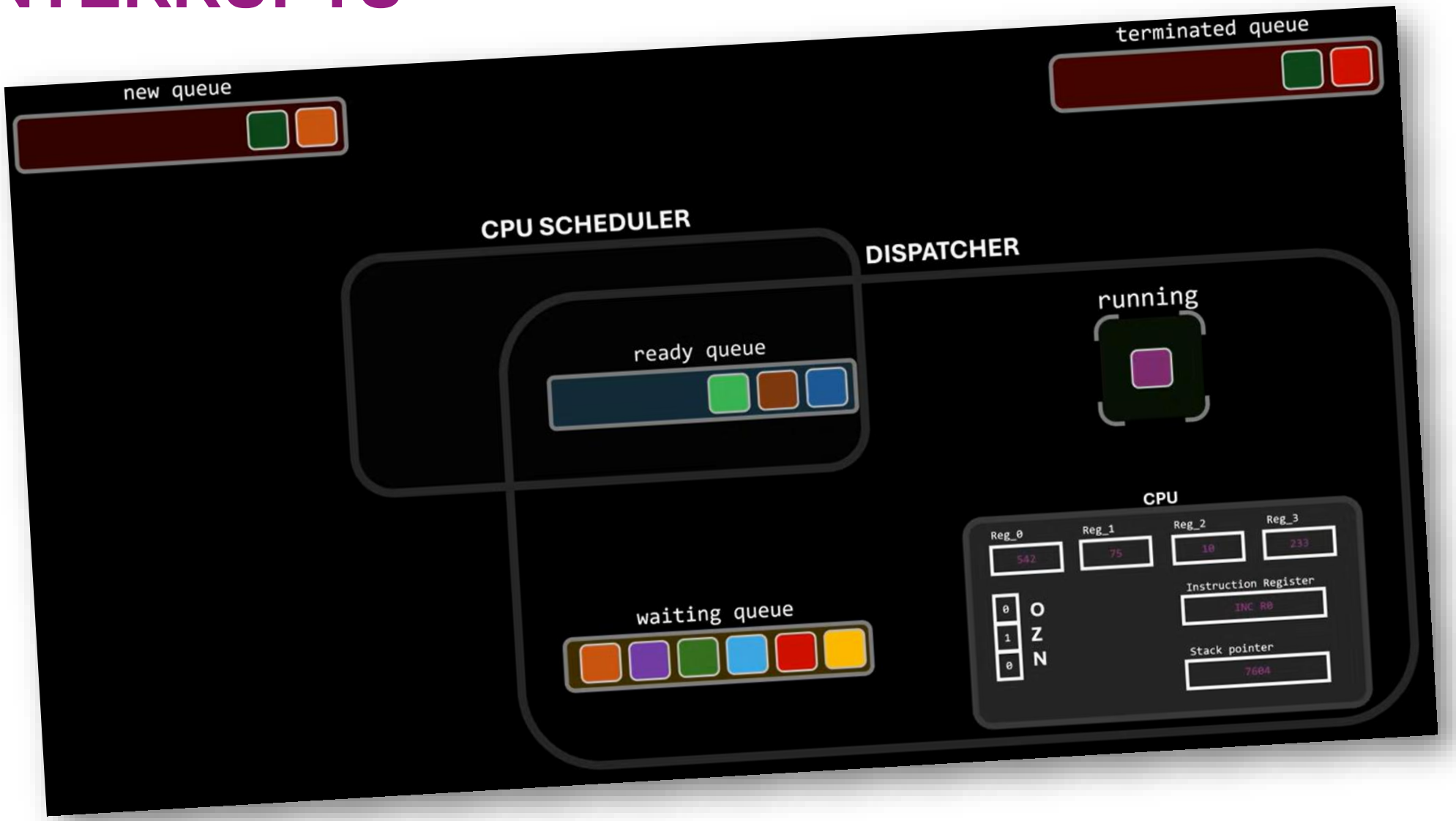
Source: https://www.youtube.com/watch?v=O2tV9q6784k

4) INTERRUPTS



Source: https://www.youtube.com/watch?v=O2tV9q6784k

4) INTERRUPTS



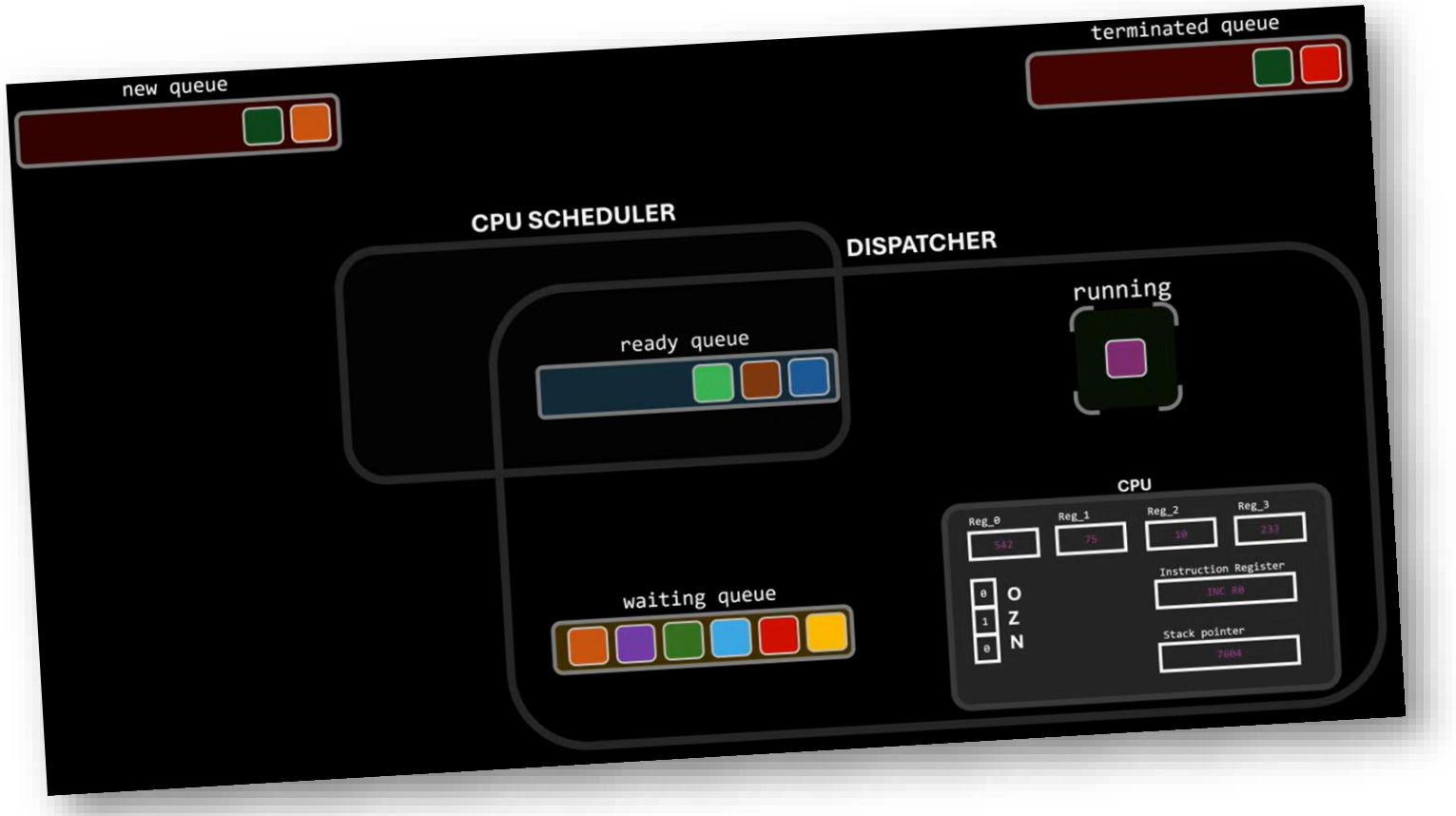
Source: https://www.youtube.com/watch?v=O2tV9q6784k

4) INTERRUPTS

The **scheduler** determines which process should use the CPU next, by managing the ready queue according to a specific scheduling policy

WHILE

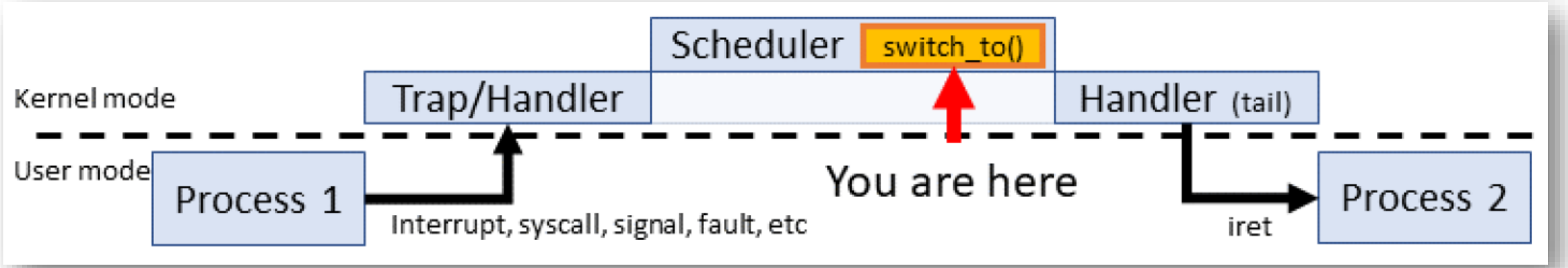
The **dispatcher** is responsible for the execution of the decision made by the scheduler.



Source: https://www.youtube.com/watch?v=O2tV9q6784k

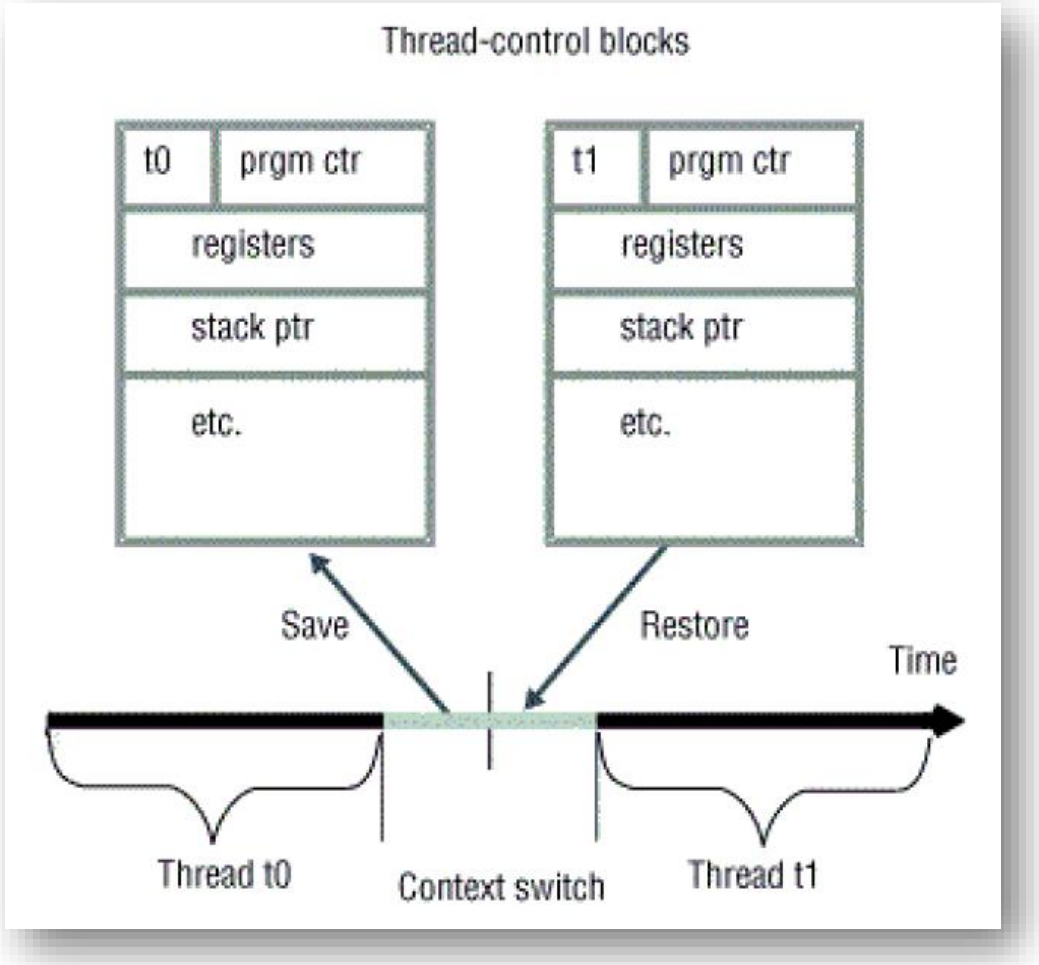
4) INTERRUPTS

- The scheduler changes the context in between the processes running in the CPU:



4) INTERRUPTS

- The context switch happen between two processes/threads running in the CPU:

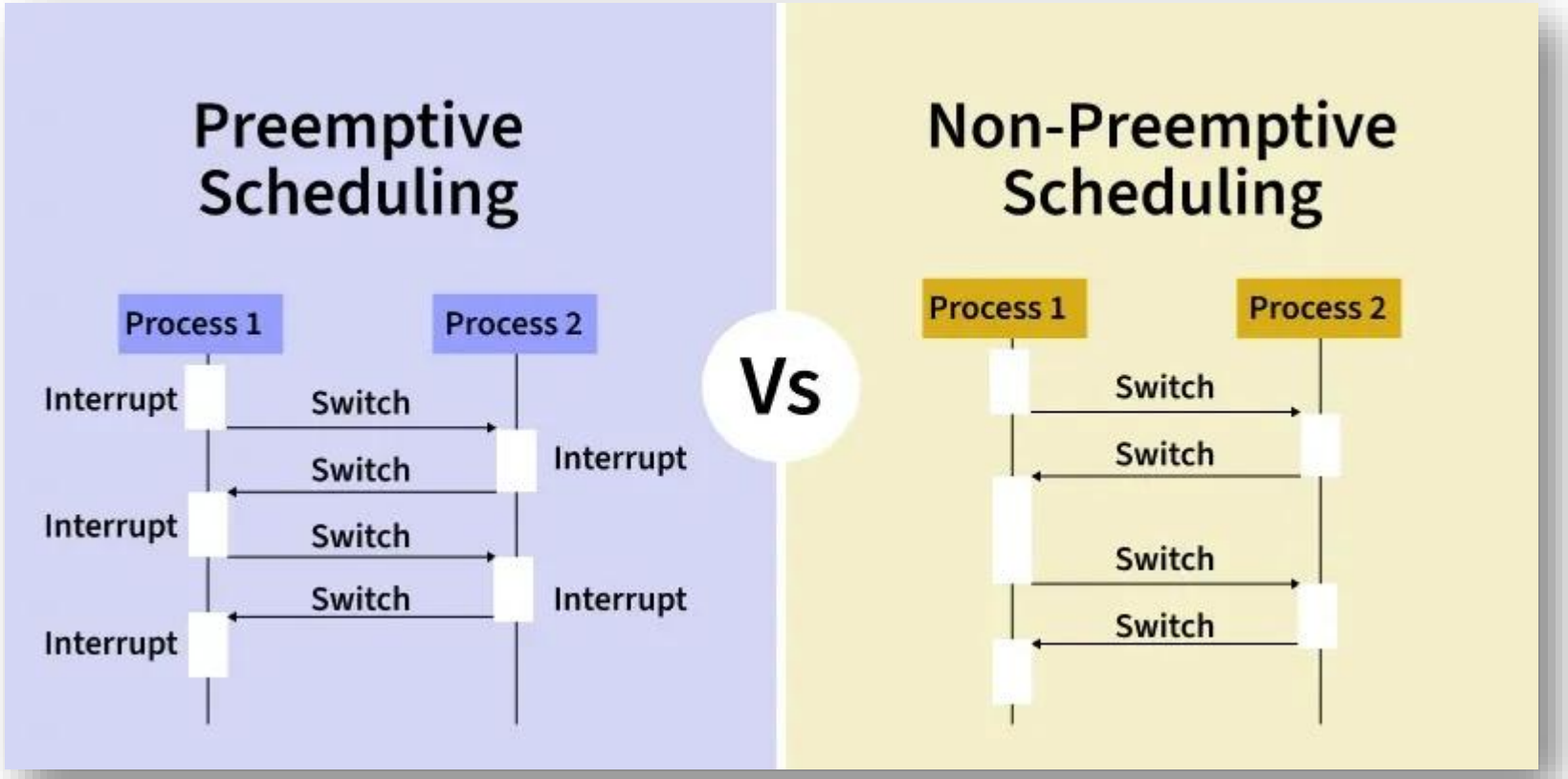


5) COOPERATIVE SCHEDULING

- **Cooperative multitasking**, also known as **non-preemptive multitasking**, is a style of computer multitasking in which the operating system never initiates a context switch from a running process to another process. Instead, in order to run multiple applications concurrently, **processes voluntarily yield control periodically** or when **idle** or **logically blocked**. This type of multitasking is called cooperative because all programs must cooperate for the scheduling scheme to work.

Source: (Wikipedia, ” Cooperative multitasking”, https://en.wikipedia.org/wiki/Cooperative_multitasking, 2024)

5) COOPERATIVE SCHEDULING



Source: (GeeksforGeeks, "Preemptive and Non-Preemptive Scheduling", https://www.geeksforgeeks.org/operating-systems/preemptive-and-non-preemptive-scheduling/, 2025)

5) Cooperative Scheduling

- What is the minimum amount of information required to schedule a periodic task?
  - How often does the task need to run?
  - When did the task last run?
  - What function should be called when it is time to run the task?

```
typedef struct
{
    uint32_t    Interval;
    uint32_t    LastTick;
    void        (*Function)(void);
} Task_t;
```

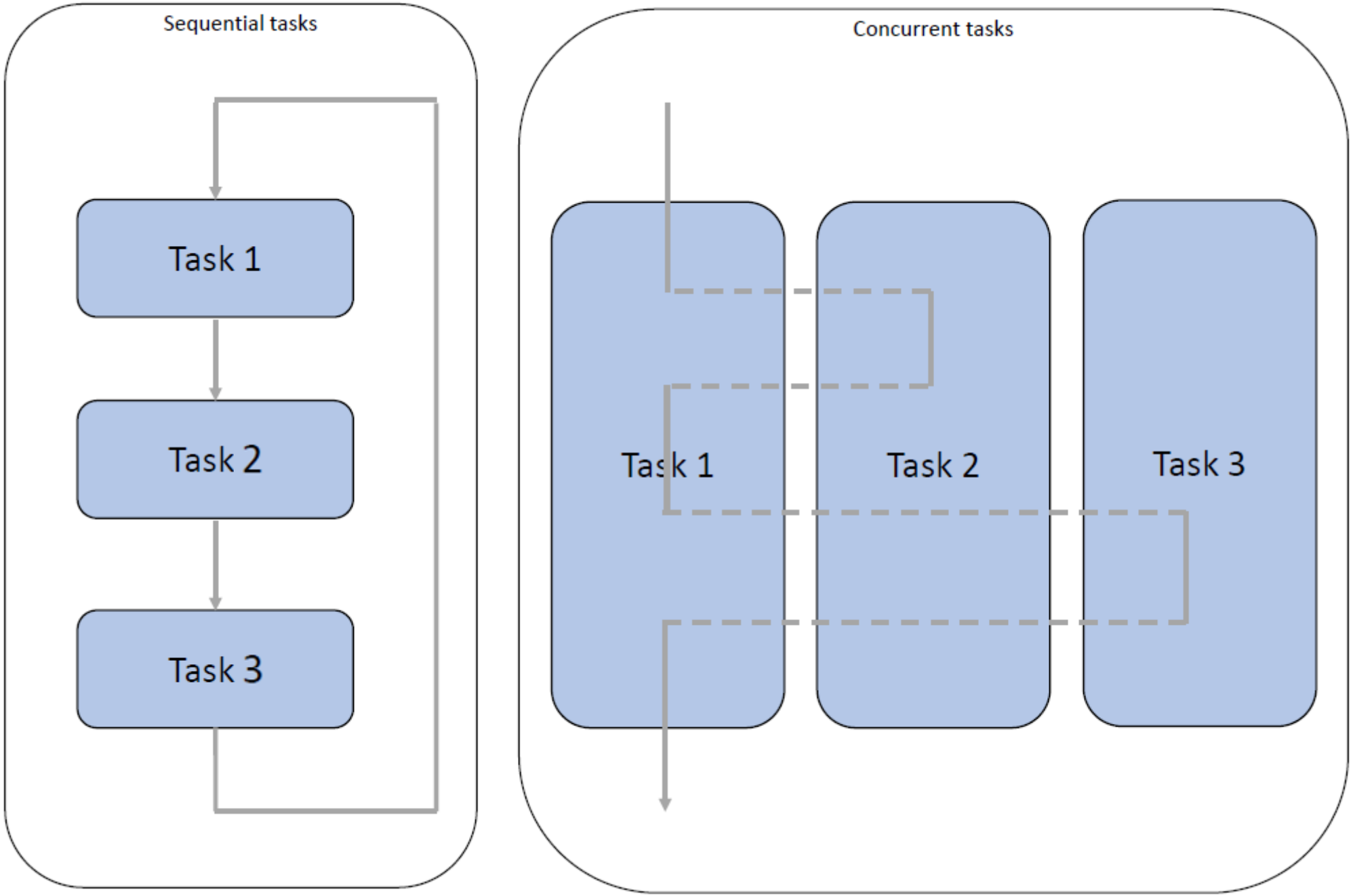
```
Task_t Tasks[] =
{
      { INTERVAL_10MS, 0, Task_10ms },
      { INTERVAL_25MS, 0, Task_25ms },
      { INTERVAL_0MS,  0, Task_idle }
};
```

while(1) { // infinite loop to assign the CPU

Tasks = taskReadQueue(); // read the tasks array from the (dynamic) waiting queue

```
for (int TaskIndex = 0; TaskIndex < NumberOfTasks; TaskIndex++)
{
  ElapsedInterval = Tick - Tasks[TaskIndex].LastTick;

  if (ElapsedInterval >= Tasks[TaskIndex].Interval)
  {
      (*Tasks[TaskIndex].Function)();      // Run the task function

      Tasks[TaskIndex].LastTick = Tick;
  }
}
```

}


Sequential tasks
Task 1
Task 2
Task 3
Concurrent tasks
Task 1
Task 2
Task 3

ANY QUESTIONS?